

```

        trace_sched_stat_runtime(curtask, delta_exec, curr->vruntime);
        cpuacct_charge(curtask, delta_exec);
        account_group_exec_runtime(curtask, delta_exec);
    }
}

```

`update_curr()` 计算了当前进程的执行时间，并且将其存放在变量 `delta_exec` 中。然后它又将运行时间传递给了 `__update_curr()`，由后者再根据当前可运行进程总数对运行时间进行加权计算。最终将上述的权重值与当前运行进程的 `vruntime` 相加。

```

/*
 * 更新当前任务的运行时统计数据。跳过不在调度类中的当前任务
 */
static inline void
__update_curr(struct cfs_rq *cfs_rq, struct sched_entity *curr,
              unsigned long delta_exec)
{
    unsigned long delta_exec_weighted;

    schedstat_set(curr->exec_max, max((u64)delta_exec, curr->exec_max));

    curr->sum_exec_runtime += delta_exec;
    schedstat_add(cfs_rq, exec_clock, delta_exec);
    delta_exec_weighted = calc_delta_fair(delta_exec, curr);

    curr->vruntime += delta_exec_weighted;
    update_min_vruntime(cfs_rq);
}

```

`update_curr()` 是由系统定时器周期性调用的，无论是在进程处于可运行态，还是被堵塞处于不可运行态。根据这种方式，`vruntime` 可以准确地测量给定进程的运行时间，而且可知道谁应该是下一个被运行的进程。

4.5.2 进程选择

在前面内容中我们的讨论中谈到若存在一个完美的多任务处理器，所有可运行进程的 `vruntime` 值将一致。但事实上我们没有找到完美的多任务处理器，因此 CFS 试图利用一个简单的规则去均衡进程的虚拟运行时间：当 CFS 需要选择下一个运行进程时，它会挑一个具有最小 `vruntime` 的进程。这其实就是 CFS 调度算法的核心：选择具有最小 `vruntime` 的任务。那么剩下的内容我们就来讨论到底是如何实现选择具有最小 `vruntime` 值的进程。

CFS 使用红黑树来组织可运行进程队列，并利用其迅速找到最小 `vruntime` 值的进程。在 Linux 中，红黑树称为 `rbtree`，它是一个自平衡二叉搜索树。我们将在第 6 章讨论自平衡二叉树以及红黑树。现在如果你还不熟悉它们，不要紧，你只需要记住红黑树是一种以树节点形式存储的数据，这些数据都会对应一个键值。我们可以通过这些键值来快速检索节点上的数据（重要的是，通过键值检索到对应节点的速度与整个树的节点规模成指数比关系）。

1. 挑选下一个任务

我们先假设，有那么一个红黑树存储了系统中所有的可运行进程，其中节点的键值便是可运行进程的虚拟运行时间。稍后我们可以看到如何生成该树，但现在我们假定已经拥有它了。CFS 调度器选取待运行的下一个进程，是所有进程中 `vruntime` 最小的那个，它对应的便是在树中最左侧的叶子节点。也就是说，你从树的根节点沿着左边的子节点向下找，一直找到叶子节点，你便找到了其 `vruntime` 值最小的那个进程。（再说一次，如果你不熟悉二叉搜索树，不用担心，只要知道它用来加速寻找过程即可）CFS 的进程选择算法可简单总结为“运行 `rbtree` 树中最左边叶子节点所代表的那个进程”。实现这一过程的函数是 `__pick_next_entity()`，它定义在文件 `kernel/sched_fair.c` 中：

```
static struct sched_entity *__pick_next_entity(struct cfs_rq *cfs_rq)
{
    struct rb_node *left = cfs_rq->rb_leftmost;

    if (!left)
        return NULL;

    return rb_entry(left, struct sched_entity, run_node);
}
```

注意 `__pick_next_entity()` 函数本身并不会遍历树找到最左叶子节点，因为该值已经缓存在 `rb_leftmost` 字段中。虽然红黑树让我们可以很有效地找到最左叶子节点（ $O(\text{树的高度})$ 等于树节点总数的 $O(\log n)$ ，这是平衡树的优势），但是更容易的做法是把最左叶子节点缓存起来。这个函数的返回值便是 CFS 调度选择的下一个运行进程。如果该函数返回值是 `NULL`，那么表示没有最左叶子节点，也就是说树中没有任何节点了。这种情况下，表示没有可运行进程，CFS 调度器便选择 `idle` 任务运行。

2. 向树中加入进程

现在，我们来看 CFS 如何将进程加入 `rbtree` 中，以及如何缓存最左叶子节点。这一切发生在进程变为可运行状态（被唤醒）或者是通过 `fork()` 调用第一次创建进程时——在第 3 章我们讨论过它。`enqueue_entity()` 函数实现了这一目的：

```
static void
enqueue_entity(struct cfs_rq *cfs_rq, struct sched_entity *se, int flags)
{
    /*
     * 通过调用 update_curr(), 在更新 min_vruntime 之前先更新规范化的 vruntime
     */
    if (!(flags & ENQUEUE_WAKEUP) || (flags & ENQUEUE_MIGRATE))
        se->vruntime += cfs_rq->min_vruntime;

    /*
     * 更新“当前任务”的运行统计数据
     */
    update_curr(cfs_rq);
}
```

```

    account_entity_enqueue(cfs_rq, se);

    if (flags & ENQUEUE_WAKEUP) {
        place_entity(cfs_rq, se, 0);
        enqueue_sleeper(cfs_rq, se);
    }

    update_stats_enqueue(cfs_rq, se);
    check_spread(cfs_rq, se);
    if (se != cfs_rq->curr)
        __enqueue_entity(cfs_rq, se);
}

```

该函数更新运行时间和其他一些统计数据，然后调用 `__enqueue_entity()` 进行繁重的插入操作，把数据项真正插入到红黑树中：

```

/* 把一个调度实体插入红黑树中 */
static void __enqueue_entity(struct cfs_rq *cfs_rq, struct sched_entity *se)
{
    struct rb_node **link = &cfs_rq->tasks_timeline.rb_node;
    struct rb_node *parent = NULL;
    struct sched_entity *entry;
    s64 key = entity_key(cfs_rq, se);
    int leftmost = 1;
    /* 在红黑树中查找合适的位置 */
    while (*link) {
        parent = *link;
        entry = rb_entry(parent, struct sched_entity, run_node);
        /*
         * 我们并不关心冲突。具有相同键值的节点呆在一起
         */
        if (key < entity_key(cfs_rq, entry)) {
            link = &parent->rb_left;
        } else {
            link = &parent->rb_right;
            leftmost = 0;
        }
    }

    /*
     * 维护一个缓存，其中存放树最左叶子节点（也就是最常使用的）
     */
    if (leftmost)
        cfs_rq->rb_leftmost = &se->run_node;

    rb_link_node(&se->run_node, parent, link);
    rb_insert_color(&se->run_node, &cfs_rq->tasks_timeline);
}

```

我们来看看上述函数，while() 循环中遍历树以寻找合适的匹配键值，该值就是被插入进程的 `vruntime`。平衡二叉树的基本规则是，如果键值小于当前节点的键值，则需转向树的左分支；相反如果大于当前节点的键值，则转向右分支。如果一旦走过右边分支，哪怕一次，也说明插入的进程不会是最左节点，因此可以设置 `leftmost` 为 0。如果一直都是向左移动，那么 `leftmost` 维持 1，这说明我们有一个新的最左节点，并且可以更新缓存——设置 `rb_leftmost` 指向被插入的进程。当我们沿着一个方向和一个没有子节的节点比较后：`link` 如果这时是 `NULL`，循环随之终止。当退出循环后，接着在父节点上调用 `rb_link_node()`，以使得新插入的进程成为其子节点。最后函数 `rb_insert_color()` 更新树的自平衡相关属性。关于着色问题，我们放在第 6 章讨论。

3. 从树中删除进程

最后我们看看 CFS 是如何从红黑树中删除进程的。删除动作发生在进程堵塞（变为不可运行态）或者终止时（结束运行）：

```
static void
dequeue_entity(struct cfs_rq *cfs_rq, struct sched_entity *se, int sleep)
{
    /*
     * 更新“当前任务”的运行统计数据
     */
    update_curr(cfs_rq);

    update_stats_dequeue(cfs_rq, se);
    clear_buddies(cfs_rq, se);

    if (se != cfs_rq->curr)
        __dequeue_entity(cfs_rq, se);
    account_entity_dequeue(cfs_rq, se);
    update_min_vruntime(cfs_rq);

    /*
     * 在更新 min_vruntime 之后对调度实体进行规范化，因为更新可以指向“->curr”项，我们需要
     * 在规范化的位置反映这一变化
     */
    if (!sleep)
        se->vruntime -= cfs_rq->min_vruntime;
}
```

和给红黑树添加进程一样，实际工作是由辅助函数 `__dequeue_entity()` 完成的。

```
static void __dequeue_entity(struct cfs_rq *cfs_rq, struct sched_entity *se)
{
    if (cfs_rq->rb_leftmost == &se->run_node) {
        struct rb_node *next_node;

        next_node = rb_next(&se->run_node);
        cfs_rq->rb_leftmost = next_node;
    }

    rb_erase(&se->run_node, &cfs_rq->tasks_timeline);
}
```

从红黑树中删除进程要容易得多。因为 rbtree 实现了 `rb_erase()` 函数，它可完成所有工作。该函数的剩下工作是更新 `rb_leftmost` 缓存。如果要删除的进程是最左节点，那么该函数要调用 `rb_next()` 按顺序遍历，找到谁是下一个节点，也就是当前最左节点被删除后，新的最左节点。

4.5.3 调度器入口

进程调度的主要入口点是函数 `schedule()`，它定义在文件 `kernel/sched.c` 中。它正是内核其他部分用于调用进程调度器的入口：选择哪个进程可以运行，何时将其投入运行。`Schedule()` 通常都需要和一个具体的调度类相关联，也就是说，它会找到一个最高优先级的调度类——后者需要有自己的可运行队列，然后问后者谁才是下一个该运行的进程。知道了这个背景，就不会吃惊 `schedule()` 函数为何实现得如此简单。该函数中唯一重要的事情是（要连这个都没有，那这个函数真是乏味得不用介绍啦），它会调用 `pick_next_task()`（也定义在文件 `kernel/sched.c` 中）。`pick_next_task()` 会以优先级为序，从高到低，依次检查每一个调度类，并且从最高优先级的调度类中，选择最高优先级的进程：

```
/*
 * 挑选最高优先级的任务
 */
static inline struct task_struct *
pick_next_task(struct rq *rq)
{
    const struct sched_class *class;
    struct task_struct *p;

    /*
     * 优化：我们知道如果所有任务都在公平类中，那么我们就可以直接调用那个函数
     */
    if (likely(rq->nr_running == rq->cfs.nr_running)) {
        p = fair_sched_class.pick_next_task(rq);
        if (likely(p))
            return p;
    }
    class = sched_class_highest;
    for ( ; ; ) {
        p = class->pick_next_task(rq);
        if (p)
            return p;

        /*
         * 永不会为 NULL，因为 idle 类总会返回非 NULL 的 p
         */
        class = class->next;
    }
}
```

注意该函数开始部分的优化。因为 CFS 是普通进程的调度类，而系统运行的绝大多数进程都是普通进程，因此这里有一个小技巧用来加速选择下一个 CFS 提供的进程，前提是所有可运行进程数量等于 CFS 类对应的可运行进程数（这样就说明所有的可运行进程都是 CFS 类的）。

该函数的核心是 `for()` 循环，它以优先级为序，从最高的优先级类开始，遍历了每一个调度

类。每一个调度类都实现了 `pick_next_task()` 函数，它会返回指向下一个可运行进程的指针，或者没有时返回 `NULL`。我们会从第一个返回非 `NULL` 值的类中选择下一个可运行进程。CFS 中 `pick_next_task()` 实现会调用 `pick_next_entity()`，而该函数会再来调用我们前面内容中讨论过的 `__pick_next_entity()` 函数。

4.5.4 睡眠和唤醒

休眠（被阻塞）的进程处于一个特殊的不可执行状态。这点非常重要，如果没有这种特殊状态的话，调度程序就可能选出一个本不愿意被执行的进程，更糟糕的是，休眠就必须以轮询的方式实现了。进程休眠有多种原因，但肯定都是为了等待一些事件。事件可能是一段时间从文件 I/O 读更多数据，或者是某个硬件事件。一个进程还有可能在尝试获取一个已被占用的内核信号量时被迫进入休眠（这部分在第 9 章中加以讨论）。休眠的一个常见原因就是文件 I/O——如进程对一个文件执行了 `read()` 操作，而这需要从磁盘里读取。还有，进程在获取键盘输入的时候也需要等待。无论哪种情况，内核的操作都相同：进程把自己标记成休眠状态，从可执行红黑树中移出，放入等待队列，然后调用 `schedule()` 选择和执行一个其他进程。唤醒的过程刚好相反：进程被设置为可执行状态，然后再从等待队列中移到可执行红黑树中。

在第 3 章里曾经讨论过，休眠有两种相关的进程状态：`TASK_INTERRUPTIBLE` 和 `TASK_UNINTERRUPTIBLE`。它们的唯一区别是处于 `TASK_UNINTERRUPTIBLE` 的进程会忽略信号，而处于 `TASK_INTERRUPTIBLE` 状态的进程如果接收到一个信号，会被提前唤醒并响应该信号。两种状态的进程位于同一个等待队列上，等待某些事件，不能够运行。

1. 等待队列

休眠通过等待队列进行处理。等待队列是由等待某些事件发生的进程组成的简单链表。内核用 `wake_queue_head_t` 来代表等待队列。等待队列可以通过 `DECLARE_WAITQUEUE()` 静态创建，也可以由 `init_waitqueue_head()` 动态创建。进程把自己放入等待队列中并设置成不可执行状态。当与等待队列相关的事件发生的时候，队列上的进程会被唤醒。为了避免产生竞争条件，休眠和唤醒的实现不能有纰漏。

针对休眠，以前曾经使用过一些简单的接口。但那些接口会带来竞争条件：有可能导致在判定条件变为真后，进程却开始了休眠，那样就会使进程无限期地休眠下去。所以，在内核中进行休眠的推荐操作就相对复杂了一些：

```
/* 'q' 是我们希望休眠的等待队列 */
DEFINE_WAIT(wait);

add_wait_queue(q, &wait);
while (!condition) { /* 'condition' 是我们在等待的事件 */
    prepare_to_wait(&q, &wait, TASK_INTERRUPTIBLE);
    if (signal_pending(current))
        /* 处理信号 */
        schedule();
}
finish_wait(&q, &wait);
```

进程通过执行下面几个步骤将自己加入到一个等待队列中：

1) 调用宏 `DEFINE_WAIT()` 创建一个等待队列的项。

2) 调用 `add_wait_queue()` 把自己加入到队列中。该队列会在进程等待的条件满足时唤醒它。当然我们必须在其他地方撰写相关代码，在事件发生时，对等待队列执行 `wake_up()` 操作。

3) 调用 `prepare_to_wait()` 方法将进程的状态变更为 `TASK_INTERRUPTIBLE` 或 `TASK_UNINTERRUPTIBLE`。而且该函数如果有必要的话会将进程加回到等待队列，这是在接下来的循环遍历中所需要的。

4) 如果状态被设置为 `TASK_INTERRUPTIBLE`，则信号唤醒进程。这就是所谓的伪唤醒（唤醒不是因为事件的发生），因此检查并处理信号。

5) 当进程被唤醒的时候，它会再次检查条件是否为真。如果是，它就退出循环；如果不是，它再次调用 `schedule()` 并一直重复这步操作。

6) 当条件满足后，进程将自己设置为 `TASK_RUNNING` 并调用 `finish_wait()` 方法把自己移出等待队列。

如果在进程开始休眠之前条件就已经达成了，那么循环会退出，进程不会存在错误地进入休眠的倾向。需要注意的是，内核代码在循环体内常常需要完成一些其他的任务，比如，它可能在调用 `schedule()` 之前需要释放掉锁，而在这以后再重新获取它们，或者响应其他事件。

函数 `inotify_read()`，位于文件 `fs/notify/inotify/inotify_user.c` 中，负责从通知文件描述符中读取信息，它的实现无疑是等待队列的一个典型用法：

```
static ssize_t inotify_read(struct file *file, char __user *buf,
                           size_t count, loff_t *pos)
{
    struct fsnotify_group *group;
    struct fsnotify_event *kevent;
    char __user *start;
    int ret;
    DEFINE_WAIT(wait);

    start = buf;
    group = file->private_data;

    while (1) {
        prepare_to_wait(&group->notification_waitq,
                        &wait,
                        TASK_INTERRUPTIBLE);

        mutex_lock(&group->notification_mutex);
        kevent = get_one_event(group, count);
        mutex_unlock(&group->notification_mutex);

        if (kevent) {
            ret = PTR_ERR(kevent);
            if (IS_ERR(kevent))
                break;
            ret = copy_event_to_user(group, kevent, buf);
            fsnotify_put_event(kevent);
            if (ret < 0)
```



```

        break;
        buf += ret;
        count -= ret;
        continue;
    }

    ret = -EAGAIN;
    if (file->f_flags & O_NONBLOCK)
        break;
    ret = -EINTR;
    if (signal_pending(current))
        break;

    if (start != buf)
        break;

    schedule();
}
finish_wait(&group->notification_waitq, &wait);

if (start != buf && ret != -EFAULT)
    ret = buf - start;
return ret;
}

```

这个函数遵循了我们例子中的使用模式，主要的区别是它在 `while` 循环中检查了状态，而不是在 `while` 循环条件语句中。原因是该条件的检测更复杂些，而且需要获得锁。也正因为如此，循环退出是通过 `break` 完成的。

2. 唤醒

唤醒操作通过函数 `wake_up()` 进行，它会唤醒指定的等待队列上的所有进程。它调用函数 `try_to_wake_up()`，该函数负责将进程设置为 `TASK_RUNNING` 状态，调用 `enqueue_task()` 将此进程放入红黑树中，如果被唤醒的进程优先级比当前正在执行的进程的优先级高，还要设置 `need_resched` 标志。通常哪段代码促使等待条件达成，它就要负责随后调用 `wake_up()` 函数。举例来说，当磁盘数据到来时，VFS 就要负责对等待队列调用 `wake_up()`，以便唤醒队列中等待这些数据的进程。

关于休眠有一点需要注意，存在虚假的唤醒。有时候进程被唤醒并不是因为它所等待的条件达成了才需要用一个循环处理来保证它等待的条件真正达成。图 4-1 描述了每个调度程序状态之间的关系。

4.6 抢占和上下文切换

上下文切换，也就是从一个可执行进程切换到另一个可执行进程，由定义在 `kernel/sched.c` 中的 `context_switch()` 函数负责处理。每当一个新的进程被选出来准备投入运行的时候，`schedule()` 就会调用该函数。它完成了两项基本的工作：

- 调用声明在 `<asm/mmu_context.h>` 中的 `switch_mm()`，该函数负责把虚拟内存从上一个进

程映射切换到新进程中。

- 调用声明在 `<asm/system.h>` 中的 `switch_to()`，该函数负责从上一个进程的处理器状态切换到新进程的处理器状态。这包括保存、恢复栈信息和寄存器信息，还有其他任何与体系结构相关的状态信息，都必须以每个进程为对象进行管理和保存。

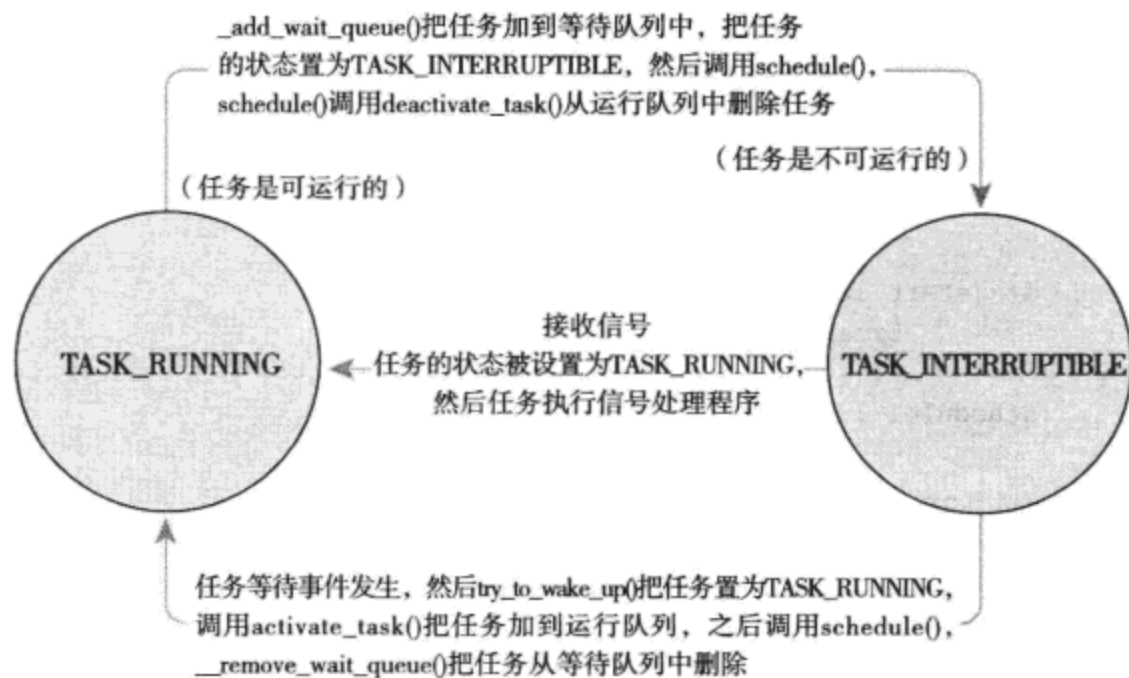


图 4-1 休眠和唤醒

内核必须知道在什么时候调用 `schedule()`。如果仅靠用户程序代码显式地调用 `schedule()`，它们可能就会永远地执行下去。相反，内核提供了一个 `need_resched` 标志来表明是否需要重新执行一次调度（见表 4-1）。当某个进程应该被抢占时，`scheduler_tick()` 就会设置这个标志；当一个优先级高的进程进入可执行状态的时候，`try_to_wake_up()` 也会设置这个标志，内核检查该标志，确认其被设置，调用 `schedule()` 来切换到一个新的进程。该标志对于内核来讲是一个信息，它表示有其他进程应当被运行了，要尽快调用调度程序。

表 4-1 用于访问和操作 `need_resched` 的函数

函 数	目 的
<code>set_tsk_need_resched()</code>	设置指定进程中的 <code>need_resched</code> 标志
<code>clear_tsk_need_resched()</code>	清除指定进程中的 <code>need_resched</code> 标志
<code>need_resched()</code>	检查 <code>need_resched</code> 标志的值，如果被设置就返回真，否则返回假

再返回用户空间以及从中断返回的时候，内核也会检查 `need_resched` 标志。如果已被设置，内核会在继续执行之前调用调度程序。

每个进程都包含一个 `need_resched` 标志，这是因为访问进程描述符内的数值要比访问一个全局变量快（因为 `current` 宏速度很快并且描述符通常都在高速缓存中）。在 2.2 以前的内核版本中，该标志曾经是一个全局变量。2.2 到 2.4 版内核中它在 `task_struct` 中。而在 2.6 版中，它被移到 `thread_info` 结构体里，用一个特别的标志变量中的一位来表示。

4.6.1 用户抢占

内核即将返回用户空间的时候，如果 `need_resched` 标志被设置，会导致 `schedule()` 被调用，此时就会发生用户抢占。在内核返回用户空间的时候，它知道自己是安全的，因为既然它可以继续去执行当前进程，那么它当然可以再去选择一个新的进程去执行。所以，内核无论是在中断处理程序还是在系统调用后返回，都会检查 `need_resched` 标志。如果它被设置了，那么，内核会选择另一个（更合适的）进程投入运行。从中断处理程序或系统调用返回的返回路径都是跟体系结构相关的，在 `entry.S`（此文件不仅包含内核入口部分的程序，内核退出部分的相关代码也在其中）文件中通过汇编语言来实现。

简而言之，用户抢占在以下情况时产生：

- 从系统调返回用户空间时。
- 从中断处理程序返回用户空间时。

4.6.2 内核抢占

与其他大部分的 Unix 变体和其他大部分的操作系统不同，Linux 完整地支持内核抢占。在不支持内核抢占的内核中，内核代码可以一直执行，到它完成为止。也就是说，调度程序没有办法在一个内核级的任务正在执行的时候重新调度——内核中的各任务是以协作方式调度的，不具备抢占性。内核代码一直要执行到完成（返回用户空间）或明显的阻塞为止。在 2.6 版的内核中，内核引入了抢占能力；现在，只要重新调度是安全的，内核就可以在任何时间抢占正在执行的任务。

那么，什么时候重新调度才是安全的呢？只要没有持有锁，内核就可以进行抢占。锁是非抢占区域的标志。由于内核是支持 SMP 的，所以，如果没有持有锁，正在执行的代码就是可重新导入的，也就是可以抢占的。

为了支持内核抢占所做的第一处变动，就是为每个进程的 `thread_info` 引入 `preempt_count` 计数器。该计数器初始值为 0，每当使用锁的时候数值加 1，释放锁的时候数值减 1。当数值为 0 的时候，内核就可执行抢占。从中断返回内核空间的时候，内核会检查 `need_resched` 和 `preempt_count` 的值。如果 `need_resched` 被设置，并且 `preempt_count` 为 0 的话，这说明有一个更为重要的任务需要执行并且可以安全地抢占，此时，调度程序就会被调用。如果 `preempt_count` 不为 0，说明当前任务持有锁，所以抢占是不安全的。这时，内核就会像通常那样直接从中断返回当前执行进程。如果当前进程持有的所有的锁都被释放了，`preempt_count` 就会重新为 0。此时，释放锁的代码会检查 `need_resched` 是否被设置。如果是的话，就会调用调度程序。有些内核代码需要允许或禁止内核抢占，相关内容会在第 9 章讨论。

如果内核中的进程被阻塞了，或它显式地调用了 `schedule()`，内核抢占也会显式地发生。这种形式的内核抢占从来都是受支持的，因为根本无须额外的逻辑来保证内核可以安全地被抢占。如果代码显式地调用了 `schedule()`，那么它应该清楚自己是可以安全地被抢占的。

内核抢占会发生在：

- 中断处理程序正在执行，且返回内核空间之前。

- 内核代码再一次具有可抢占性的时候。
- 如果内核中的任务显式地调用 `schedule()`。
- 如果内核中的任务阻塞（这同样也会导致调用 `schedule()`）。

4.7 实时调度策略

Linux 提供了两种实时调度策略：`SCHED_FIFO` 和 `SCHED_RR`。而普通的、非实时的调度策略是 `SCHED_NORMAL`。借助调度类的框架，这些实时策略并不被完全公平调度器来管理，而是被一个特殊的实时调度器管理。具体的实现定义在文件 `kernel/sched_rt.c` 中，在接下来的内容中我们将讨论实时调度策略和算法。

`SCHED_FIFO` 实现了一种简单的、先入先出的调度算法：它不使用时间片。处于可运行状态的 `SCHED_FIFO` 级的进程会比任何 `SCHED_NORMAL` 级的进程都先得到调度。一旦一个 `SCHED_FIFO` 级进程处于可执行状态，就会一直执行，直到它自己受阻塞或显式地释放处理器为止；它不基于时间片，可以一直执行下去。只有更高优先级的 `SCHED_FIFO` 或者 `SCHED_RR` 任务才能抢占 `SCHED_FIFO` 任务。如果有两个或者更多的同优先级的 `SCHED_FIFO` 级进程，它们会轮流执行，但是依然只有在它们愿意让出处理器时才会退出。只要有 `SCHED_FIFO` 级进程在执行，其他级别较低的进程就只能等待它变为不可运行态后才有机会执行。

`SCHED_RR` 与 `SCHED_FIFO` 大体相同，只是 `SCHED_RR` 级的进程在耗尽事先分配给它的时间后就不能再继续执行了。也就是说，`SCHED_RR` 是带有时间片的 `SCHED_FIFO`——这是一种实时轮流调度算法。当 `SCHED_RR` 任务耗尽它的时间片时，在同一优先级的其他实时进程被轮流调度。时间片只用来重新调度同一优先级的进程。对于 `SCHED_FIFO` 进程，高优先级总是立即抢占低优先级，但低优先级进程决不能抢占 `SCHED_RR` 任务，即使它的时间片耗尽。

这两种实时算法实现的都是静态优先级。内核不为实时进程计算动态优先级。这能保证给定优先级别的实时进程总能抢占优先级比它低的进程。

Linux 的实时调度算法提供了一种软实时工作方式。软实时的含义是，内核调度进程，尽力使进程在它的限定时间到来前运行，但内核不保证总能满足这些进程的要求。相反，硬实时系统保证在一定条件下，可以满足任何调度的要求。Linux 对于实时任务的调度不做任何保证。虽然不能保证硬实时工作方式，但 Linux 的实时调度算法的性能还是很不错的。2.6 版的内核可以满足严格的时间要求。

实时优先级范围从 0 到 `MAX_RT_PRIO` 减 1。默认情况下，`MAX_RT_PRIO` 为 100——所以默认的实时优先级范围是从 0 到 99。`SCHED_NORMAL` 级进程的 `nice` 值共享了这个取值空间；它的取值范围是从 `MAX_RT_PRIO` 到 $(\text{MAX_RT_PRIO} + 40)$ 。也就是说，在默认情况下，`nice` 值从 -20 到 +19 直接对应的是从 100 到 139 的实时优先级范围。

4.8 与调度相关的系统调用

Linux 提供了一个系统调用族，用于管理与调度程序相关的参数。这些系统调用可以用来操作和处理进程优先级、调度策略及处理器绑定，同时还提供了显式地将处理器交给其他进程的机制。

许多书籍（还有友善的 man 帮助文件）都提供了这些系统调用（它们都包含在 C 库中，没用什么太多的封装，基本上只调用了系统调用而已）的说明。表 4-2 列举了这些系统调用并给出了简短的说明。第 5 章会讨论它们是如何实现的。

表 4-2 与调度相关的系统调用

系统调用	描 述
<code>nice()</code>	设置进程的 nice 值
<code>sched_setscheduler()</code>	设置进程的调度策略
<code>sched_getscheduler()</code>	获取进程的调度策略
<code>sched_setparam()</code>	设置进程的实时优先级
<code>sched_getparam()</code>	获取进程的实时优先级
<code>sched_get_priority_max()</code>	获取实时优先级的最大值
<code>sched_get_priority_min()</code>	获取实时优先级的最小值
<code>sched_rr_get_interval()</code>	获取进程的时间片值
<code>sched_setaffinity()</code>	设置进程的处理器的亲和力
<code>sched_getaffinity()</code>	获取进程的处理器的亲和力
<code>sched_yield()</code>	暂时让出处理器

4.8.1 与调度策略和优先级相关的系统调用

`sched_setscheduler()` 和 `sched_getscheduler()` 分别用于设置和获取进程的调度策略和实时优先级。与其他的系统调用相似，它们的实现也是由许多参数检查、初始化和清理构成的。其实最重要的工作在于读取或改写进程 `task_struct` 的 `policy` 和 `rt_priority` 的值。

`sched_setparam()` 和 `sched_getparam()` 分别用于设置和获取进程的实时优先级。这两个系统调用获取封装在 `sched_param` 特殊结构体的 `rt_priority` 中。`sched_get_priority_max()` 和 `sched_get_priority_min()` 分别用于返回给定调度策略的最大和最小优先级。实时调度策略的最大优先级是 `MAX_USER_RT_PRIO` 减 1，最小优先级等于 1。

对于一个普通的进程，`nice()` 函数可以将给定进程的静态优先级增加一个给定的量。只有超级用户才能在调用它时使用负值，从而提高进程的优先级。`nice()` 函数会调用内核的 `set_user_nice()` 函数，这个函数会设置进程的 `task_struct` 的 `static_prio` 和 `prio` 值。

4.8.2 与处理器绑定有关的系统调用

Linux 调度程序提供强制的处理器绑定（processor affinity）机制。也就是说，虽然它尽力通过一种软的（或者说自然的）亲和性试图使进程尽量在同一个处理器上运行，但它也允许用户强制指定“这个进程无论如何都必须在这些处理器上运行”。这种强制的亲和性保存在进程 `task_struct` 的 `cpus_allowed` 这个位掩码标志中。该掩码标志的每一位对应一个系统可用的处理器。默认情况下，所有的位都被设置，进程可以在系统中所有可用的处理器上执行。用户可以通过 `sched_setaffinity()` 设置不同的一个或几个位组合的位掩码，而调用 `sched_getaffinity()` 则返回当前的 `cpus_allowed` 位掩码。

内核提供的强制处理器绑定的方法很简单。首先，当处理进行第一次创建时，它继承了其父进程的相关掩码。由于父进程运行在指定处理器上，子进程也运行在相应处理器上。其次，当处理器绑定关系改变时，内核会采用“移植线程”把任务推到合法的处理器上。最后，加载平衡器只把任务拉到允许的处理器上，因此，进程只运行在指定处理器上，对处理器的指定是由该进程描述符的 `cpus_allowed` 域设置的。

4.8.3 放弃处理器时间

Linux 通过 `sched_yield()` 系统调用，提供了一种让进程显式地将处理器时间让给其他等待执行进程的机制。它是通过将进程从活动队列中（因为进程正在执行，所以它肯定位于此队列当中）移到过期队列中实现的。由此产生的效果不仅抢占了该进程并将其放入优先级队列的最后面，还将其放入过期队列中——这样能确保在一段时间内它都不会再被执行了。由于实时进程不会过期，所以属于例外。它们只被移动到你优先级队列的最后面（不会放到过期队列中）。在 Linux 的早期版本中，`sched_yield()` 的语义有所不同，进程只会被放置到优先级队列的末尾，放弃的时间往往不会太长。现在，应用程序甚至内核代码在调用 `sched_yield()` 前，应该仔细考虑是否真的希望放弃处理器时间。

内核代码为了方便，可以直接调用 `yield()`，先要确定给定进程确实处于可执行状态，然后再调用 `sched_yield()`。用户空间的应用程序直接使用 `sched_yield()` 系统调用就可以了。

4.9 小结

进程调度程序是内核重要的组成部分，因为运行着的进程首先在使用计算机（至少在我们大多数人看来）。然而，满足进程调度的各种需要绝不是轻而易举的：很难找到“一刀切”的算法，既适合众多的可运行进程，又具有可伸缩性，还能在调度周期和吞吐量之间求得平衡，同时还满足各种负载的需求。不过，Linux 内核的新 CFS 调度程序尽量满足了各个方面的需求，并以较完善的可伸缩性和新颖的方法提供了最佳的解决方案。

前面的章节覆盖了进程管理的相关内容，本章则考察了进程调度所遵循的基本原理、具体实现、调度算法以及目前 Linux 内核所使用的接口。第 5 章将涵盖内核提供给运行进程的主要接口——系统调用。



第⑤章

系统调用

在现代操作系统中，内核提供了用户进程与内核进行交互的一组接口。这些接口让应用程序受限地访问硬件设备，提供了创建新进程并与已有进程进行通信的机制，也提供了申请操作系统其他资源的能力。这些接口在应用程序和内核之间扮演了使者的角色，应用程序发出各种请求，而内核负责满足这些请求（或者无法满足时返回一个错误）。实际上提供这些接口主要是为了保证系统稳定可靠，避免应用程序恣意妄行。

5.1 与内核通信

系统调用在用户空间进程和硬件设备之间添加了一个中间层。该层主要作用有三个。首先，它为用户空间提供了一种硬件的抽象接口。举例来说，当需要读写文件的时候，应用程序就可以不去管磁盘类型和介质，甚至不用去管文件所在的文件系统到底是哪种类型。第二，系统调用保证了系统的稳定和安全。作为硬件设备和应用程序之间的中间人，内核可以基于权限、用户类型和其他一些规则对需要进行的访问进行裁决。举例来说，这样可以避免应用程序不正确地使用硬件设备，窃取其他进程的资源，或做出其他危害系统的事情。第三，在第3章中曾经提到过，每个进程都运行在虚拟系统中，而在用户空间和系统的其余部分提供这样一层公共接口，也是出于这种考虑。如果应用程序可以随意访问硬件而内核又对此一无所知的话，几乎就没法实现多任务和虚拟内存，当然也不可能实现良好的稳定性和安全性。在Linux中，系统调用是用户空间访问内核的唯一手段；除异常和陷入外，它们是内核唯一的合法入口。实际上，其他的像设备文件和`/proc`之类的方式，最终也还是要通过系统调用进行访问的。而有趣的是，Linux提供的系统调用却比大部分操作系统都少得多^①。本章重点强调Linux系统调用的规则和实现方法。

5.2 API、POSIX 和 C 库

一般情况下，应用程序通过为用户空间实现的应用编程接口(API)而不是直接通过系统调用来编程。这点很重要，因为应用程序使用的这种编程接口实际上并不需要和内核提供的系统调用对应。一个API定义了一组应用程序使用的编程接口。它们可以实现成一个系统调用，也可以通过调用多个系统调用来实现，而完全不使用任何系统调用也不存在问题。实际上，API可以在各种不同的操作系统上实现，给应用程序提供完全相同的接口，而它们本身在这些系统上的实现却可能迥异。图5-1给出POSIX、API、C库以及系统调用之间的关系。

① x86系统上大概有250个系统调用（每种体系结构都会定义一些独特的系统调用）。尽管有些系统还没有完全公布所有的系统调用，但据估计某些操作系统的系统调用数有上千个。

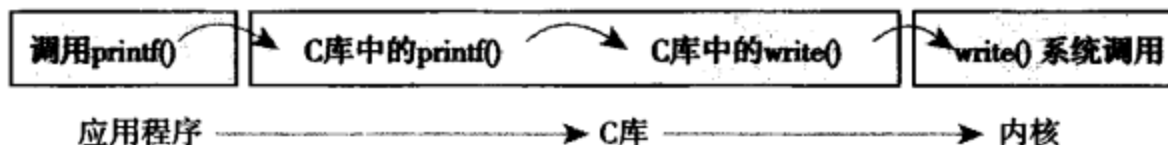


图 5-1 调用 `printf()` 函数时，应用程序、C 库和内核之间的关系

在 Unix 世界中，最流行的应用编程接口是基于 POSIX 标准的。从纯技术的角度看，POSIX 是由 IEEE^①的一组标准组成，其目标是提供一套大体上基于 Unix 的可移植操作系统标准。在应用场合，Linux 尽力与 POSIX 和 SUSv3 兼容。

POSIX 是说明 API 和系统调用之间关系的一个极好例子。在大多数 Unix 系统上，根据 POSIX 定义的 API 函数和系统调用之间有着直接关系。实际上，POSIX 标准就是仿照早期 Unix 系统的接口建立的。另一方面，许多操作系统，像微软的 Windows，尽管是非 Unix 系统，也提供了与 POSIX 兼容的库。

Linux 的系统调用像大多数 Unix 系统一样，作为 C 库的一部分提供。C 库实现了 Unix 系统的主要 API，包括标准 C 库函数和系统调用接口。所有的 C 程序都可以使用 C 库，而由于 C 语言本身的特点，其他语言也可以很方便地把它们封装起来使用。此外，C 库提供了 POSIX 的绝大部分 API。

从程序员的角度看，系统调用无关紧要，他们只需要跟 API 打交道就可以了。相反，内核只跟系统调用打交道；库函数及应用程序是怎么使用系统调用，不是内核所关心的。但是，内核必须时刻牢记系统调用所有潜在的用途，并保证它们有良好的通用性和灵活性。

关于 Unix 的接口设计有一句格言“提供机制而不是策略”。换句话说，Unix 的系统调用抽象出了用于完成某种确定的目的的函数。至于这些函数怎么用完全不需要内核去关心^②。

5.3 系统调用

要访问系统调用（在 Linux 中常称作 `syscall`），通常通过 C 库中定义的函数调用来进行。它们通常都需要定义零个、一个或几个参数（输入）而且可能产生一些副作用^③，例如，写某个文件或向给定的指针拷贝数据等。系统调用还会通过一个 `long` 类型^④的返回值来表示成功或者错误。通常，但也不绝对，用一个负的返回值来表明错误。返回一个 0 值通常（当然仍不是绝对的）表明成功。系统调用在出现错误的时候 C 库会把错误码写入 `errno` 全局变量。通过调用 `perror()` 库函数，可以把该变量翻译成用户可以理解的错误字符串。

当然，系统调用最终具有一种明确的操作。例如 `getpid()` 系统调用，根据定义它会返回当前

① IEEE (eye-triple-E) 是电气和电子工程师协会。这是一个非盈利组织，它涉及的技术领域非常广泛，并且对许多重要标准负责。欲了解更多信息，请访问 <http://www.ieee.org>。

② 区别对待机制 (mechanism) 和策略 (policy) 是 Unix 设计中的一大亮点。大部分的编程问题都可以被切割成两个部分：“需要提供什么功能”（机制）和“怎样实现这些功能”（策略）。如果由程序中的独立部分分别负责机制和策略的实现，那么开发软件就更容易，也更容易适应不同的需求。——译者注

③ 注意这里用的是可能。尽管绝大部分调用都会产生某种副作用（就是说，它们会使系统的状态发生某种变化），但还是有一些系统调用，如 `getpid()`，仅仅返回一些内核数据。

④ 使用 `long` 类型是为了与 64 位的硬件体系结构保持兼容。

进程的 PID。内核中它的实现非常简单：

```
SYSCALL_DEFINE0(getpid)
{
    return task_tgid_vnr(current); // returns current->tgid
}
```

注意，定义中并没有规定它要如何实现。内核必须提供系统调用所希望完成的功能，但它完全可以按照自己预期的方式去实现，只要最后的结果正确就行了。当然，上面的系统调用太简单，也没有什么更多的实现手段^①。

SYSCALL_DEFINE0 只是一个宏，它定义一个无参数的系统调用（因此这里为数字 0），展开后的代码如下：

```
asmlinkage long sys_getpid(void)
```

我们看一下如何定义系统调用。首先，注意函数声明中的 `asmlinkage` 限定词，这是一个编译指令，通知编译器仅从栈中提取该函数的参数。所有的系统调用都需要这个限定词。其次，函数返回 `long`。为了保证 32 位和 64 位系统的兼容，系统调用在用户空间和内核空间有不同的返回值类型，在用户空间为 `int`，在内核空间为 `long`。最后，注意系统调用 `get_pid()` 在内核中被定义成 `sys_getpid()`。这是 Linux 中所有系统调用都应该遵守的命名规则，系统调用 `bar()` 在内核中也实现为 `sys_bar()` 函数。

5.3.1 系统调用号

在 Linux 中，每个系统调用被赋予一个系统调用号。这样，通过这个独一无二的号就可以关联系统调用。当用户空间的进程执行一个系统调用的时候，这个系统调用号就用来指明到底是要执行哪个系统调用；进程不会提及系统调用的名称。

系统调用号相当重要，一旦分配就不能再有任何变更，否则编译好的应用程序就会崩溃。此外，如果一个系统调用被删除，它所占用的系统调用号也不允许被回收利用，否则，以前编译过的代码会调用这个系统调用，但事实上却调用的是另一个系统调用。Linux 有一个“未实现”系统调用 `sys_ni_syscall()`，它除了返回 `-ENOSYS` 外不做任何其他工作，这个错误号就是专门针对无效的系统调用而设的。虽然很罕见，但如果一个系统调用被删除，或者变得不可用，这个函数就要负责“填补空缺”。

内核记录了系统调用表中的所有已注册过的系统调用的列表，存储在 `sys_call_table` 中。每一种体系结构中，都明确定义了这个表，在 x86-64 中，它定义于 `arch/i386/kernel/syscall_64.c` 文件中。这个表为每一个有效的系统调用指定了唯一的系统调用号。

5.3.2 系统调用的性能

Linux 系统调用比其他许多操作系统执行得要快。Linux 很短的上下文切换时间是一个重要

^① 你可能会想为什么 `getpid()` 返回的是 `tgid`（即线程组 ID）呢？原因在于，对于普通进程来说，`TGID` 和 `PID` 相等。对于线程来说，同一线程组内的所有线程其 `TGID` 都相等。这使得这些线程能够调用 `getpid()` 并得到相同的 `PID`。

原因，进出内核都被优化得简洁高效。另外一个原因是系统调用处理程序和每个系统调用本身也都非常简洁。

5.4 系统调用处理程序

用户空间的程序无法直接执行内核代码。它们不能直接调用内核空间中的函数，因为内核驻留在受保护的地址空间上。如果进程可以直接在内核的地址空间上读写的话，系统的安全性和稳定性将不复存在。

所以，应用程序应该以某种方式通知系统，告诉内核自己需要执行一个系统调用，希望系统切换到内核态，这样内核就可以代表应用程序在内核空间执行系统调用。

通知内核的机制是靠软中断实现的：通过引发一个异常来促使系统切换到内核态去执行异常处理程序。此时的异常处理程序实际上就是系统调用处理程序。在 x86 系统上预定义的软中断是中断号 128，通过 `int $0x80` 指令触发该中断。这条指令会触发一个异常导致系统切换到内核态并执行第 128 号异常处理程序，而该程序正是系统调用处理程序。这个处理程序名字起得很贴切，叫 `system_call()`。它与硬件体系结构紧密相关^①，x86-64 的系统上在 `entry_64.S` 文件中用汇编语言编写。最近，x86 处理器增加了一条叫做 `sysenter` 的指令。与 `int` 中断指令相比，这条指令提供了更快、更专业的陷入内核执行系统调用的方式。对这条指令的支持很快被加入内核。且不管系统调用处理程序被如何调用，用户空间引起异常或陷入内核就是一个重要的概念。

5.4.1 指定恰当的系统调用

因为所有的系统调用陷入内核的方式都一样，所以仅仅是陷入内核空间是不够的。因此必须把系统调用号一并传给内核。在 x86 上，系统调用号是通过 `eax` 寄存器传递给内核的。在陷入内核之前，用户空间就把相应系统调用所对应的号放入 `eax` 中。这样系统调用处理程序一旦运行，就可以从 `eax` 中得到数据。其他体系结构上的实现也都类似。

`system_call()` 函数通过将给定的系统调用号与 `NR_syscalls` 做比较来检查其有效性。如果它大于或者等于 `NR_syscalls`，该函数就返回 `-ENOSYS`。否则，就执行相应的系统调用：

```
call *sys_call_table(,%rax,8)
```

由于系统调用表中的表项是以 64 位（8 字节）类型存放的，所以内核需要将给定的系统调用号乘以 4，然后用所得的结果在该表中查询其位置。在 x86-32 系统上，代码很类似，只是用 4 代替 8，参见图 5-2。

5.4.2 参数传递

除了系统调用号以外，大部分系统调用都还需要一些外部的参数输入。所以，在发生陷入的时候，应该把这些参数从用户空间传给内核。最简单的办法就是像传递系统调用号一样，把这些

^① 下面许多关于系统调用处理程序的描述都是针对 x86 版本的。但不用担心，所有体系结构上的实现都类似。

参数也存放在寄存器里。在 x86-32 系统上, `ebx`、`ecx`、`edx`、`esi` 和 `edi` 按照顺序存放前五个参数。需要六个或六个以上参数的情况不多见, 此时, 应该用一个单独的寄存器存放指向所有这些参数在用户空间地址的指针。

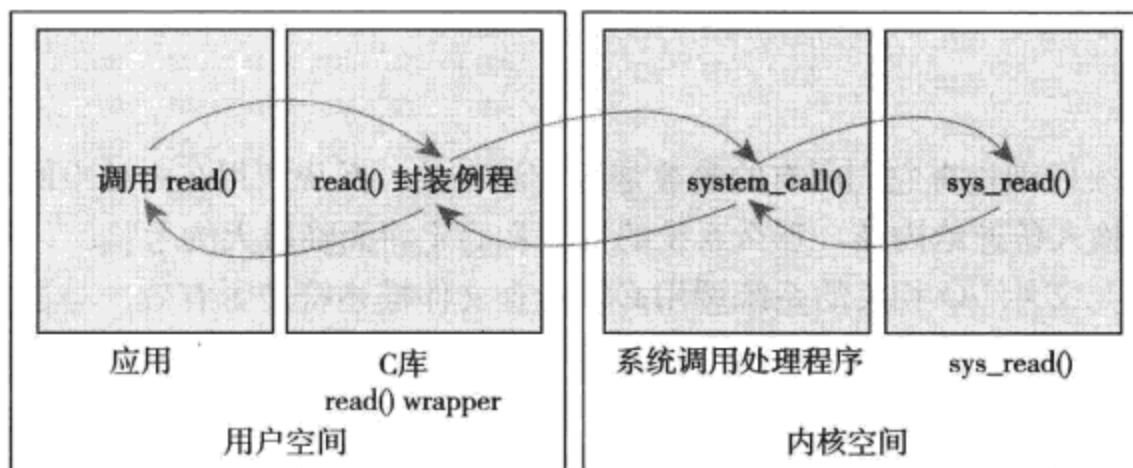


图 5-2 调用系统调用处理程序以执行一个系统调用

给用户空间的返回值也通过寄存器传递。在 x86 系统上, 它存放在 `eax` 寄存器中。

5.5 系统调用的实现

实际上, 一个 Linux 的系统调用在实现时并不需要太关心它和系统调用处理程序之间的关系。给 Linux 添加一个新的系统调用是件相对容易的工作。怎样设计和实现一个系统调用是难题所在, 而把它加到内核里却无须太多周折。让我们关注一下实现一个新的 Linux 系统调用所需的步骤。

5.5.1 实现系统调用

实现一个新的系统调用的第一步是决定它的用途。它要做些什么? 每个系统调用都应该有一个明确的用途。在 Linux 中不提倡采用多用途的系统调用 (一个系统调用通过传递不同的参数值来选择完成不同的工作)。`ioctl()` 就是一个很好的例子, 告诉了我们不应当去做什么。

新系统调用的参数、返回值和错误码又该是什么呢? 系统调用的接口应该力求简洁, 参数尽可能少。系统调用的语义和行为非常关键; 因为应用程序依赖于它们, 所以它们应力求稳定, 不做改动。设想一下, 如果功能多次改变会怎样。新的功能是否可以追加到系统调用亦或是否某个改变将需要一个全新的函数? 是否可以容易地修订错误而不用破坏向后兼容? 很多系统调用提供了标志参数以确保向前兼容。标志并不是用来让单个系统调用具有多个不同的行为 (如前所述, 这是不允许的), 而是为了即使增加新的功能和选项, 也不破坏向后兼容或不需要增加新的系统调用。

设计接口的时候要尽量为将来多做考虑。你是不是对函数做了不必要的限制? 系统调用设计得越通用越好。不要假设这个系统调用现在怎么用将来也一定就是这么用。系统调用的目的可能不变, 但它的用法却可能改变。这个系统调用可移植吗? 别对机器的字节长度和字节序做假设。第 19 章将讨论这个话题。要确保不对系统调用做错误的假设, 否则将来这个调用就可能会崩溃。

记住 Unix 的格言：“提供机制而不是策略”。

当你写一个系统调用的时候，要时刻注意可移植性和健壮性，不但要考虑当前，还要为将来做打算。基本的 Unix 系统调用经受住了时间的考验；它们中的很大一部分到现在都还和 30 年前一样适用和有效。

5.5.2 参数验证

系统调用必须仔细检查它们所有的参数是否合法有效。系统调用在内核空间执行，如果任由用户将不合法的输入传递给内核，那么系统的安全和稳定将面临极大的考验。

举例来说，与文件 I/O 相关的系统调用必须检查文件描述符是否有效。与进程相关的函数必须检查提供的 PID 是否有效。必须检查每个参数，保证它们不但合法有效，而且正确。进程不应当让内核去访问那些它无权访问的资源。

最重要的一种检查就是检查用户提供的指针是否有效。试想，如果一个进程可以给内核传递指针而又无须检查，那么它就可以给出一个它根本就没有访问权限的指针，哄骗内核去为它拷贝本不允许它访问的数据，如原本属于其他进程的数据或者不可读的映射数据。在接收一个用户空间的指针之前，内核必须保证：

- 指针指向的内存区域属于用户空间。进程决不能哄骗内核去读内核空间的数据。
- 指针指向的内存区域在进程的地址空间里。进程决不能哄骗内核去读其他进程的数据。
- 如果是读，该内存应被标记为可读；如果是写，该内存应被标记为可写；如果是可执行，该内存被标记为可执行。进程决不能绕过内存访问限制。

内核提供了两个方法来完成必须的检查和内核空间与用户空间之间数据的来回拷贝。注意，内核无论何时都不能轻率地接受来自用户空间的指针！这两个方法中必须经常有一个被使用。

为了向用户空间写入数据，内核提供了 `copy_to_user()`，它需要三个参数。第一个参数是进程空间中的目的内存地址，第二个是内核空间内的源地址，最后一个参数是需要拷贝的数据长度（字节数）。

为了从用户空间读取数据，内核提供了 `copy_from_user()`，它和 `copy_to_user()` 相似。该函数把第二个参数指定的位置上的数据拷贝到第一个参数指定的位置上，拷贝的数据长度由第三个参数决定。

如果执行失败，这两个函数返回的都是没能完成拷贝的数据的字节数。如果成功，则返回 0。当出现上述错误时，系统调用返回标准 `-EFAULT`。

让我们以一个既用了 `copy_from_user()` 又用了 `copy_to_user()` 的系统调用作例子进行考察。这个系统调用 `silly_copy()` 毫无实际用处，它从第一个参数里拷贝数据到第二个参数。这种用途让人无法理解，它毫无必要地让内核空间作为中转站，把用户空间的数据从一个位置复制到另外一个位置。但它却能演示出上述函数的用法。

```
/*
 * silly_copy 没有实际价值的系统调用，它把 len 字节的数据从 'src' 拷贝到 'dst'，毫无理由地让内核空
 * 间作为中转站。但这的确是个好例子
 */
```

```

SYSCALL_DEFINE3(silly_copy,
                unsigned long *, src,
                unsigned long *, dst,
                unsigned long len)
{
    unsigned long buf;
    /* 将用户地址空间中的 src 拷贝进 buf */
    if (copy_from_user(&buf, src, len))
        return -EFAULT;
    /* 将 buf 拷贝进用户地址空间中的 dst */
    if (copy_to_user(dst, &buf, len))
        return -EFAULT;
    /* 返回拷贝的数据量 */
    return len;
}

```

注意，`copy_to_user()` 和 `copy_from_user()` 都有可能引起阻塞。当包含用户数据的页被换出到硬盘上而不是在物理内存上的时候，这种情况就会发生。此时，进程就会休眠，直到缺页处理程序将该页从硬盘重新换回物理内存。

最后一项检查针对是否有合法权限。在老版本的 Linux 内核中，需要超级用户权限的系统调用才可以通过调用 `suser()` 函数这个标准动作来完成检查。这个函数只能检查用户是否为超级用户；现在它已经被一个更细粒度的“权能”机制代替。新的系统允许检查针对特定资源的特殊权限。调用者可以使用 `capable()` 函数来检查是否有权能对指定的资源进行操作，如果它返回非 0 值，调用者就有权进行操作，返回 0 则无权操作。举个例子，`capable(CAP_SYS_NICE)` 可以检查调用者是否有权改变其他进程的 nice 值。默认情况下，属于超级用户的进程拥有所有权利而非超级用户没有任何权利。例如，下面是 `reboot()` 系统调用，注意，第一步是如何确保调用进程具有 `CAP_SYS_REBOOT` 权能。如果那样一个条件语句被删除，任何进程都可以启动系统了。

```

SYSCALL_DEFINE4(reboot,
                int, magic1,
                int, magic2,
                unsigned int, cmd,
                void __user *, arg)
{
    char buffer[256];

    /* 我们只信任启动系统的系统管理员 */
    if (!capable(CAP_SYS_BOOT))
        return -EPERM;

    /* 为了安全起见，我们需要 "magic" 参数 */
    if (magic1 != LINUX_REBOOT_MAGIC1 ||
        (magic2 != LINUX_REBOOT_MAGIC2 &&
         magic2 != LINUX_REBOOT_MAGIC2A &&
         magic2 != LINUX_REBOOT_MAGIC2B &&
         magic2 != LINUX_REBOOT_MAGIC2C))
        return -EINVAL;

    /* 当未设置 pm_power_off 时，请不要试图让 power_off 的代码看起来像是可以停机，而应该采用更简单的方式 */

```

```

    if ((cmd == LINUX_REBOOT_CMD_POWER_OFF) && !pm_power_off)
        cmd = LINUX_REBOOT_CMD_HALT;

    lock_kernel();
    switch (cmd) {
    case LINUX_REBOOT_CMD_RESTART:
        kernel_restart(NULL);
        break;

    case LINUX_REBOOT_CMD_CAD_ON:
        C_A_D = 1;
        break;
    case LINUX_REBOOT_CMD_CAD_OFF:
        C_A_D = 0;
        break;

    case LINUX_REBOOT_CMD_HALT:
        kernel_halt();
        unlock_kernel();
        do_exit(0);
        break;

    case LINUX_REBOOT_CMD_POWER_OFF:
        kernel_power_off();
        unlock_kernel();
        do_exit(0);
        break;

    case LINUX_REBOOT_CMD_RESTART2:
        if (strncpy_from_user(&buffer[0], arg, sizeof(buffer) - 1) < 0) {
            unlock_kernel();
            return -EFAULT;
        }
        buffer[sizeof(buffer) - 1] = '\0';

        kernel_restart(buffer);
        break;

    default:
        unlock_kernel();
        return -EINVAL;
    }
    unlock_kernel();
    return 0;
}

```

参见 `<linux/capability.h>`，其中包含一份所有这些权能和其对应的权限的列表。

5.6 系统调用上下文

在第3章中曾经讨论过，内核在执行系统调用的时候处于进程上下文。`current` 指针指向当

前任务，即引发系统调用的那个进程。

在进程上下文中，内核可以休眠（比如在系统调用阻塞或显式调用 `schedule()` 的时候）并且可以被抢占。这两点都很重要。首先，能够休眠说明系统调用可以使用内核提供的绝大部分功能。在第 7 章中我们会看到，休眠的能力会给内核编程带来极大便利[⊖]。在进程上下文中能够被抢占其实表明，像用户空间内的进程一样，当前的进程同样可以被其他进程抢占。因为新的进程可以使用相同的系统调用，所以必须小心，保证该系统调用是可重入的。当然，这也是在对称多处理中必须同样关心的问题。关于可重入的保护涵盖在第 9 章和第 10 章中。

当系统调用返回的时候，控制权仍然在 `system_call()` 中，它最终会负责切换到用户空间，并让用户进程继续执行下去。

5.6.1 绑定一个系统调用的最后步骤

当编写完一个系统调用后，把它注册成一个正式的系统调用是件琐碎的工作：

1) 首先，在系统调用表的最后加入一个表项。每种支持该系统调用的硬件体系都必须做这样的工作（大部分的系统调用都针对所有的体系结构）。从 0 开始算起，系统调用在该表中的位置就是它的系统调用号。如第 10 个系统调用分配到的系统调用号为 9。

2) 对于所支持的各种体系结构，系统调用号都必须定义于 `<asm/unistd.h>` 中。

3) 系统调用必须被编译进内核映像（不能被编译成模块）。这只要把它放进 `kernel/` 下的一个相关文件中就可以了，比如 `sys.c`，它包含了各种各样的系统调用。

让我们通过一个虚构的系统调用 `foo()` 来仔细观察一下这些步骤。首先，我们要把 `sys_foo` 加入到系统调用表中去。对于大多数体系结构来说，该表位于 `entry.s` 文件中，形式如下：

```
ENTRY(sys_call_table)
    .long sys_restart_syscall    /* 0 */
    .long sys_exit
    .long sys_fork
    .long sys_read
    .long sys_write
    .long sys_open                /* 5 */
    ...

    .long sys_eventfd2
    .long sys_epoll_create1
    .long sys_dup3                /* 330 */
    .long sys_pipe2
    .long sys_inotify_init1
    .long sys_preadv
    .long sys_pwritev
```

⊖ 中断处理程序不能休眠，这使得中断处理程序所能进行的操作较之运行在进程上下文中的系统调用所能进行的操作受到了极大的限制。

```
.long sys_rt_tgsigqueueinfo    /* 335 */
.long sys_perf_event_open
.long sys_recvmmsg
```

我们把新的系统调用加到这个表的末尾：

```
.long sys_foo
```

虽然没有明确地指定编号，但我们加入的这个系统调用被按照次序分配给了 338 这个系统调用号。对于每种需要支持的体系结构，我们都必须将自己的系统调用加入到其系统调用表中去。每种体系结构不需要对应相同的系统调用号。系统调用号是专属于体系结构 ABI（应用程序二进制接口）的部分。通常，你需要让系统调用适应每种体系结构。你可以注意一下，每隔 5 个表项就加入一个调用号注释的习惯，这可以在查找系统调用对应的调用号时提供方便。

接下来，我们把系统调用号加入到 `<asm/unistd.h>` 中，它的格式如下：

```
/*
 * 本文件包含系统调用号
 */

#define __NR_restart_syscall    0
#define __NR_exit                1
#define __NR_fork                2
#define __NR_read                3
#define __NR_write               4
#define __NR_open                5
...
#define __NR_signalfd4           327
#define __NR_eventfd2            328
#define __NR_epoll_create1       329
#define __NR_dup3                330
#define __NR_pipe2               331
#define __NR_inotify_init1       332
#define __NR_preadv               333
#define __NR_pwritev              334
#define __NR_rt_tgsigqueueinfo   335
#define __NR_perf_event_open     336
#define __NR_recvmmsg             337
```

然后，我们在该列表中加入下面这行：

```
#define __NR_foo                338
```

最后，我们来实现 `foo()` 系统调用。无论何种配置，该系统调用都必须编译到核心的内核映像中去，所以在这个例子中我们把它放进 `kernel/sys.c` 文件中。你也可以将其放到与其功能联系最紧密的代码中去，假如它的功能与调度相关，那么你也可以把它放到 `kernel/sched.c` 中去。

```
#include <asm/page.h>
```

```
/*
 * sys_foo - 每个人喜欢的系统调用
 * 返回每个进程的内核栈大小
 */
asm linkage long sys_foo(void)
{
    return THREAD_SIZE;
}
```

就是这样！现在就可以启动内核并在用户空间调用 `foo()` 系统调用了。

5.6.2 从用户空间访问系统调用

通常，系统调用靠 C 库支持。用户程序通过包含标准头文件并和 C 库链接，就可以使用系统调用（或者调用库函数，再由库函数实际调用）。但如果你仅仅写出系统调用，glibc 库恐怕并不提供支持。

值得庆幸的是，Linux 本身提供了一组宏，用于直接对系统调用进行访问。它会设置好寄存器并调用陷入指令。这些宏是 `_syscalln()`，其中 n 的范围从 0 到 6，代表需要传递给系统调用的参数个数，这是由于该宏必须了解到底有多少参数按照什么次序压入寄存器。举个例子，`open()` 系统调用的定义是：

```
long open(const char *filename, int flags, int mode)
```

而不靠库支持，直接调用此系统调用的宏的形式为：

```
#define NR_open 5
__syscall13(long, open, const char*, filename, int, flags, int, mode)
```

这样，应用程序就可以直接使用 `open()`。

对于每个宏来说，都有 $2+2\times n$ 个参数。第一个参数对应着系统调用的返回值类型。第二个参数是系统调用的名称。再以后是按照系统调用参数的顺序排列的每个参数的类型和名称。`_NR_open` 在 `<asm/unistd.h>` 中定义，是系统调用号。该宏会被扩展成为内嵌汇编的 C 函数；由汇编语言执行前面内容中所讨论的步骤，将系统调用号和参数压入寄存器并触发软中断来陷入内核。调用 `open()` 系统调用直接把上面的宏放置在应用程序中就可以了。

让我们写一个宏来使用前面编写的 `foo()` 系统调用，然后再写出测试代码炫耀一下我们所做的努力。

```
#define __NR_foo 283
__syscall10(long, foo)

int main ()
{
    long stack_size;

    stack_size = foo ();
    printf ("The kernel stack size is %ld\n", stack_size);

    return 0;
}
```

5.6.3 为什么不通过系统调用的方式实现

前面的内容已经告诉大家，建立一个新的系统调用非常容易，但却绝不提倡这么做。的确，你应该多多练习如何给一个新的系统调用加警告与限制。通常都会有更好的办法用来代替新建一个系统调用以作实现。让我们看看采用系统调用作为实现方式的利弊和替代的方法。

建立一个新的系统调用的好处：

- 系统调用创建容易且使用方便。
- Linux 系统调用的高性能显而易见。

问题是：

- 你需要一个系统调用号，而这需要一个内核在处于开发版本的时候由官方分配给你。
- 系统调用被加入稳定内核后就被固化了，为了避免应用程序的崩溃，它的接口不允许做改动。
- 需要将系统调用分别注册到每个需要支持的体系结构中去。
- 在脚本中不容易调用系统调用，也不能从文件系统直接访问系统调用。
- 由于你需要系统调用号，因此在主内核树之外是很难维护和使用系统调用的。
- 如果仅仅进行简单的信息交换，系统调用就大材小用了。

替代方法：

实现一个设备节点，并对此实现 `read()` 和 `write()`。使用 `ioctl()` 对特定的设置进行操作或者对特定的信息进行检索。

- 像信号量这样的某些接口，可以用文件描述符来表示，因此也就可以按上述方式对其进行操作。
- 把增加的信息作为一个文件放在 `sysfs` 的合适位置。

对于许多接口来说，系统调用都被视为正确的解决之道。但 Linux 系统尽量避免每出现一种新的抽象就简单的加入一个新的系统调用。这使得它的系统调用接口简洁得令人叹为观止，也就避免了许多后悔和反对意见（系统调用再也不被使用或支持）。新系统调用增添频率很低也反映出 Linux 是一个相对较为稳定并且功能已经较为完善的操作系统。

5.7 小结

在本章，我们描述了系统调用到底是什么，它们与库函数和应用程序接口（API）有怎样的关系。然后，我们考察了 Linux 内核如何实现系统调用，以及执行系统调用的连锁反应：陷入内核，传递系统调用号和参数，执行正确的系统调用函数，并把返回值带回用户空间。

然后，我们讨论了如何增加系统调用，并提供了从用户空间调用系统调用的简单例子。整个过程相当容易！增加一个新的系统调用没有什么难的，这一过程也就是系统调用的实现过程。本书的其余部分讨论了编写规范的、最优化的、安全的系统调用所遵循的概念和内核接口规范。

最后，我们通过讨论实现系统调用的优缺点以及列举其替代方案的形式对全章内容进行了总结。

第⑥章

内核数据结构

本章将介绍几种 Linux 内核常用的内建数据结构。和其他很多大型项目一样，Linux 内核实现了这些通用数据结构，而且提倡大家在开发时重用。内核开发者应该尽可能地使用这些数据结构，而不要搞自作主张的山寨方法。在下面的内容中，我们讲述这些通用数据结构中最有用的几个：

- 链表
- 队列
- 映射
- 二叉树

本章最后还要讨论算法复杂度，以及何种规模的算法或数据结构可以相对容易地支持更大的输入集合。

6.1 链表

链表是 Linux 内核中最简单、最普通的数据结构。链表是一种存放和操作可变数量元素（常称为节点）的数据结构。链表和静态数组的不同之处在于，它所包含的元素都是动态创建并插入链表的，在编译时不必知道具体需要创建多少个元素。另外也因为链表中每个元素的创建时间各不相同，所以它们在内存中无须占用连续内存区。正是因为元素不连续地存放，所以各元素需要通过某种方式被连接在一起。于是每个元素都包含一个指向下一个元素的指针，当有元素加入链表或从链表中删除元素时，简单调整指向下一个节点的指针就可以了。

6.1.1 单向链表和双向链表

可以用一种最简单的数据结构来表示这样一个链表：

```
/* 一个链表中的一个元素 */
struct list_element {
    void *data;                /* 有效数据 */
    struct list_element *next; /* 指向下一个元素的指针 */
};
```

图 6-1 描述一个链表结构体。

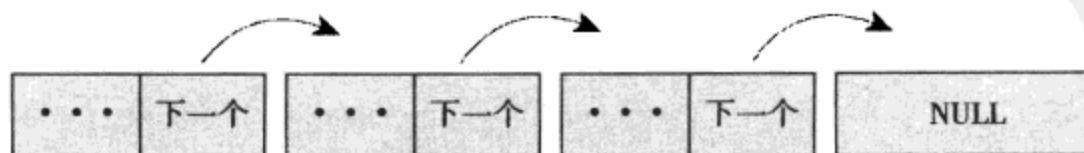


图 6-1 一个简单链表

在有些链表中，每个元素还包含一个指向前一个元素的指针，因为它们可以同时向前和向后相互连接，所以这种链表被称作双向链表。而类似于图 6-1 所示的那种只能向后连接的链表被称作单向链表。

表示双向链表的一种数据结构如下：

```
/* 一个链表中的一个元素 */
struct list_element {
    void *data;                /* 有效数据 */
    struct list_element *next; /* 指向下一个元素的指针 */
    struct list_element *prev; /* 指向前一个元素的指针 */
};
```

图 6-2 描述了一个双向链表。

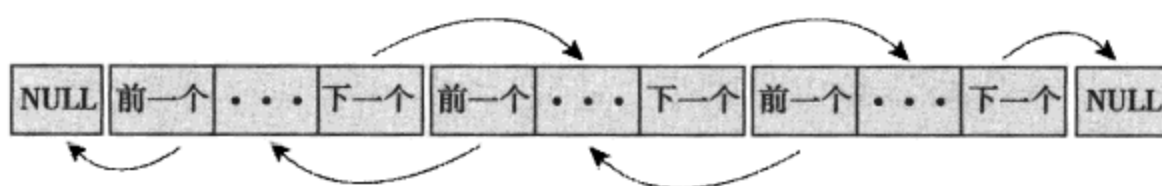


图 6-2 一个双向链表

6.1.2 环形链表

通常情况下，因为链表中最后一个元素不再有下一个元素，所以将链表尾元素中的向后指针设置为 NULL，以此表明它是链表中的最后一个元素。但在有些链表中，末尾元素并不指向特殊值，相反，它指回链表的首元素。这种链表因为首尾相连，所以被称为是环形链表。环形链表也存在双向链表和单向链表两种形式。在环形双向链表中，首节点的向前指针指向尾节点。图 6-3 和图 6-4 分别表示单向和环形双向链表。

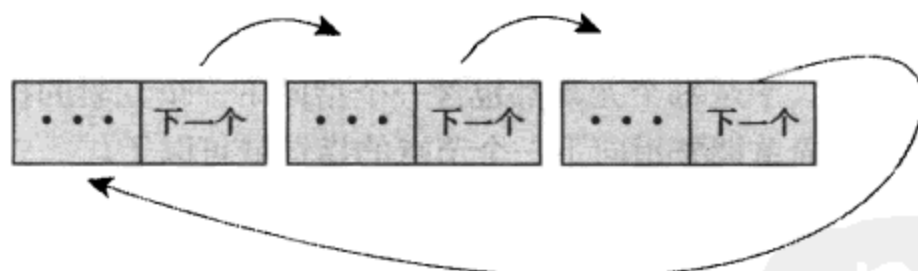


图 6-3 环形单向链表

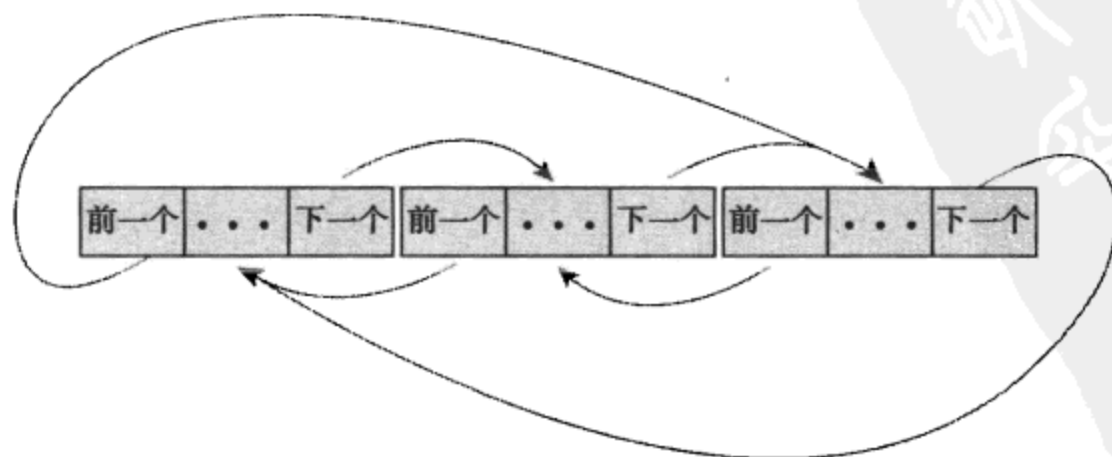


图 6-4 环形双向链表

因为环形双向链表提供了最大的灵活性，所以 Linux 内核的标准链表就是采用环形双向链表形式实现的。

6.1.3 沿链表移动

沿链表移动只能是线性移动。先访问某个元素，然后沿该元素的向后指针访问下一个元素，不断重复这个过程，就可以沿链表向后移动了。这是一种最简单的沿链表移动方法，也是最适合访问链表的方法。如果需要随机访问数据，一般不使用链表。使用链表存放数据的理想情况是，需要遍历所有数据或需要动态加入和删除数据时。

有时，首元素会用一个特殊指针表示——该指针称为头指针，利用头指针可方便、快速找到链表的“起始端”。在非环形链表里，向后指针指向 NULL 的元素是尾元素，而在环形链表里向后指针指向头元素的元素是尾元素。遍历一个链表需要线性地访问从第一个元素到最后一个元素之间的所有元素。对于双向链表来说，也可以反向遍历链表，可以从最后一个元素线性访问到第一个元素。当然还可以从链表中的指定元素开始向前和向后访问数个元素，并不一定要访问整个链表。

6.1.4 Linux 内核中的实现

相比普遍的链表实现方式（包括前面章节描述的通用方法），Linux 内核的实现可以说独树一帜。回忆早先提到的数据（或者一组数据，比如一个 struct）通过在内部添加一个指向数据的 next（或者 previous）节点指针，才能串联在链表中。比如，假定我们有一个 fox 数据结构来描述犬科动物中的一员。

```
struct fox {
    unsigned long tail_length;    /* 尾巴长度，以厘米为单位 */
    unsigned long weight;        /* 重量，以千克为单位 */
    bool          is_fantastic;   /* 这只狐狸奇妙吗？ */
};
```

存储这个结构到链表里的通常方法是在数据结构中嵌入一个链表指针，比如：

```
struct fox {
    unsigned long tail_length;    /* 尾巴长度，以厘米为单位 */
    unsigned long weight;        /* 重量，以千克为单位 */
    bool          is_fantastic;   /* 这只狐狸奇妙吗？ */
    struct fox    *next;          /* 指向下一个狐狸 */
    struct fox    *prev;          /* 指向前一个狐狸 */
};
```

Linux 内核方式与众不同，它不是将数据结构塞入链表，而是将链表节点塞入数据结构！

1. 链表数据结构

在过去，内核中有许多链表的实现，该选一个既简单、又高效的链表来统一它们了。在 2.1 内核开发系列中，首次引入了官方内核链表实现。从此内核中的所有链表现在都使用官方的链表实现了，千万不要再自己造轮子啦！

链表代码在头文件 <linux/list.h> 中声明，其数据结构很简单：

```
struct list_head {
    struct list_head *next;
    struct list_head *prev;
};
```

`next` 指针指向下一个链表节点，`prev` 指针指向前一个。然而，似乎这里还看不出它们有多大的作用。到底什么才是链表存储的具体内容呢？其实关键在于理解 `list_head` 结构是如何使用的。

```
struct fox {
    unsigned long    tail_length;    /* 尾巴长度，以厘米为单位 */
    unsigned long    weight;         /* 重量，以千克为单位 */
    bool             is_fantastic;   /* 这只狐狸奇妙吗？ */
    struct list_head list;           /* 所有 fox 结构体形成链表 */
};
```

上述结构中，`fox` 中的 `list.next` 指向下一个元素，`list.prev` 指向前一个元素。现在链表已经能用了，但是显然还不够方便。因此内核又提供了一组链表操作例程。比如 `list_add()` 方法加入一个新节点到链表中。但是，这些方法都有一个统一的特点：它们只接受 `list_head` 结构作为参数。使用宏 `container_of()` 我们可以很方便地从链表指针找到父结构中包含的任何变量。这是因为在 C 语言中，一个给定结构中的变量偏移在编译时地址就被 ABI 固定下来了。

```
#define container_of(ptr, type, member) ({ \
    const typeof( ((type *)0)->member ) *__mptr = (ptr); \
    (type *) ( (char *)__mptr - offsetof(type,member) );})
```

使用 `container_of()` 宏，我们定义一个简单的函数便可返回包含 `list_head` 的父类型结构体：

```
#define list_entry(ptr, type, member) \
    container_of(ptr, type, member)
```

依靠 `list_entry()` 方法，内核提供了创建、操作以及其他链表管理的各种例程——所有这些方法都不需要知道 `list_head` 所嵌入对象的数据结构。

2. 定义一个链表

正如看到的：`list_head` 本身其实并没有意义——它需要被嵌入到你自己的数据结构中才能生效：

```
struct fox {
    unsigned long    tail_length;    /* 尾巴长度，以厘米为单位 */
    unsigned long    weight;         /* 重量，以千克为单位 */
    bool             is_fantastic;   /* 这只狐狸奇妙吗？ */
    struct list_head list;           /* 所有 fox 结构体形成链表 */
};
```

链表需要在使用前初始化。因为多数元素都是动态创建的（也许这就是需要链表的原因），因此最常见的方式是在运行时初始化链表。

```
struct fox *red_fox;
red_fox = kmalloc(sizeof(*red_fox), GFP_KERNEL);
red_fox->tail_length = 40;
red_fox->weight = 6;
red_fox->is_fantastic = false;
INIT_LIST_HEAD(&red_fox->list);
```

如果一个结构在编译期静态创建，而你需要在其中给出一个链表的直接引用，下面是最简方式：

```
struct fox red_fox = {
    .tail_length = 40,
    .weight = 6,
    .list = LIST_HEAD_INIT(red_fox.list),
};
```

3. 链表头

前面我们展示了如何把一个现有的数据结构（这里是我们的 fox 结构体）改造成链表。

简单修改上述代码，我们的结构便可以被内核链表例程管理。但是在可以使用这些例程前，需要一个标准的索引指针指向整个链表，即链表的头指针。

内核链表实现中最杰出的特性就是：我们的 fox 节点都是无差别的——每一个都包含一个 list_head 指针，于是我们可以从任何一个节点起遍历链表，直到我们看到所有节点。这种方式确实很优美，不过有时确实也需要一个特殊指针索引到整个链表，而不从一个链表节点触发。有趣的是，这个特殊的索引节点事实上也就是一个常规的 list_head：

```
static LIST_HEAD(fox_list);
```

该函数定义并初始化了一个名为 fox_list 的链表例程，这些例程中的大多数都只接受一个或者两个参数：头节点或者头节点加上一个特殊链表节点。下面我们就具体看看这些操作例程。

6.1.5 操作链表

内核提供了一组函数来操作链表，这些函数都要使用一个或多个 list_head 结构体指针作参数。因为函数都是用 C 语言以内联函数形式实现的，所以它们的原型在文件 <linux/list.h> 中。

有趣的是，所有这些函数的复杂度都为 $O(1)$ [⊖]。这意味着，无论这些函数操作的链表大小如何，无论它们得到的参数如何，它们都在恒定时间内完成。比如，不管是对于包含 3 个元素的链表还是对于包含 3000 个元素的链表，从链表中删除一项或加入一项花费的时间都是相同的。这点可能没什么让人惊奇的，但你最好还是搞清楚其中的原因。

1. 向链表中增加一个节点

给链表增加一个节点：

```
list_add(struct list_head *new, struct list_head *head)
```

该函数向指定链表的 head 节点后插入 new 节点。因为链表是循环的，而且通常没有首尾节点的概念，所以你可以把任何一个节点当成 head。如果把“最后”一个节点当做 head 的话，那么该函数可以用来实现一个栈。

回到我们的例子，假定我们创建一个新的 struct fox，并把它加入 fox_list，那么我们这样做：

```
list_add(&f->list, &fox_list);
```

⊖ 请看本章 6.6 节，其中讨论了 $O(1)$ 算法。

把节点增加到链表尾：

```
list_add_tail(struct list_head *new, struct List_head *head)
```

该函数向指定链表的 head 节点前插入 new 节点。和 list_add() 函数类似，因为链表是环形的，所以可以把任何一个节点当做 head。如果把“第一个”元素当做 head 的话，那么该函数可以用来实现一个队列。

2. 从链表中删除一个节点

在链表中增加一个节点后，从中删除一个结点是另一个最重要的操作。从链表中删除一个结点，调用 list_del()：

```
list_del(struct list_head *entry)
```

该函数从链表中删除 entry 元素。注意，该操作并不会释放 entry 或释放包含 entry 的数据结构体所占用的内存；该函数仅仅是将 entry 元素从链表中移走，所以该函数被调用后，通常还需要再撤销包含 entry 的数据结构体和其中的 entry 项。

例如，为了删除 for 节点，我们回到前面增加节点的 fox_list：

```
list_del(&f->list)
```

注意，该函数并没有接受 fox_list 作为输入参数。它只是接受一个特定的节点，并修改其前后节点的指针，这样给定的节点就从链表中删除。代码的实现颇具启发性：

```
static inline void __list_del(struct list_head *prev, struct list_head *next)
{
    next->prev = prev;
    prev->next = next;
}

static inline void list_del(struct list_head *entry)
{
    __list_del(entry->prev, entry->next);
}
```

从链表中删除一个节点并对其重新初始化：

```
list_del_init():
list_del_init(struct list_head *entry)
```

该函数除了还需要再次初始化 entry 以外，其他和 list_del() 函数类似。这样做是因为：虽然链表不再需要 entry 项，但是还可以再次使用包含 entry 的数据结构体。

3. 移动和合并链表节点

把节点从一个链表移到另一个链表：

```
list_move(struct list_head *list, struct list_head *head)
```

该函数从一个链表中移除 list 项，然后将其加入到另一链表的 head 节点后面。

把节点从一个链表移到另一个链表的末尾：

```
list_move_tail(struct list_head *list, struct list_head *head)
```

该函数和 `list_move()` 函数一样，唯一的不同是将 `list` 项插入到 `head` 项前。

检查链表是否为空：

```
list_empty(struct list_head *head)
```

如果指定的链表为空，该函数返回非 0 值；否则返回 0。

把两个未连接的链表合并在一起：

```
list_splice(struct list_head *list, struct list_head *head)
```

该函数合并两个链表，它将 `list` 指向的链表插入到指定链表的 `head` 元素后面。

把两个未连接的链表合并在一起，并重新初始化原来的链表：

```
list_splice_init(struct list_head *list, struct list_head *head)
```

该函数和 `list_splice()` 函数一样，唯一的不同是由 `list` 指向的链表要被重新初始化。

节约两次提领 (dereference)

如果你碰巧已经得到了 `next` 和 `prev` 指针，你可以直接调用内部链表函数，从而省下一点时间（其实就是提领指针的时间）。前面讨论的所有函数其实没有做什么其他特别的操作，它仅仅是找到 `next` 和 `prev` 指针，再去调用内部函数而已。内部函数和它们的外部包装函数同名，仅仅在前面加了两条下划线。比如，可以调用 `__list_del(prev, next)` 函数代替调用 `list_del(list)` 函数。但这只有在向前和向后指针确实已经被提领过的情况下才有意义。否则，你只是在画蛇添足。请看文件 `<linux/list.h>` 中具体的接口。

6.1.6 遍历链表

现在，你已经知道了如何在内核中声明、初始化和操作一个链表。这很了不起，但如果无法访问自己的数据，这些没有任何意义。链表仅仅是个能够包含重要数据的容器；我们必须利用链表移动并访问包含我们数据的结构体。幸好，内核为我们提供了一组非常棒的接口，可以用来遍历链表和引用链表中的数据结构体。

注意 和链表操作函数不同，遍历链表的复杂度为 $O(n)$ ， n 是链表所包含的元素数目。

1. 基本方法

遍历链表最简单的方法是使用 `list_for_each()` 宏，该宏使用两个 `list_head` 类型的参数，第一个参数用来指向当前项，这是一个你必须提供的临时变量，第二个参数是需要遍历的链表的以头节点形式存在的 `list_head`（见前面的“链表头”部分）。每次遍历中，第一个参数在链表中不断移动指向下一个元素，直到链表中的所有元素都被访问为止。用法如下：

```
struct list_head *p;
list_for_each(p, list) {
    /*p 指向链表中的元素*/
}
```

好了，实话实说，其实一个指向链表结构的指针通常是无用的；我们所需要的是一个指向包含 `list_head` 的结构体的指针。比如前面 `fox` 结构体的例子，我们需要的是指向每个 `fox` 的指针，而不需要指向结构体中 `list` 成员的指针。我们可以使用前面讨论的 `list_entry()` 宏，来获得包含给定 `list_head` 的数据结构。比如：

```
struct list_head *p;
struct fox *f;

list_for_each(p, &fox_list) {
    /* f points to the structure in which the list is embedded */
    f = list_entry(p, struct fox, list);
}
```

2. 可用的方法

前面的方法虽然确实展示了 `list_head` 节点的功效，但并不优美，而且也不够灵活。所以多数内核代码采用 `list_for_each_entry()` 宏遍历链表。该宏内部也使用 `list_entry()` 宏，但简化了遍历过程：

```
list_for_each_entry(pos, head, member)
```

这里 `pos` 是一个指向包含 `list_head` 节点对象的指针，可将它看做是 `list_entry` 宏的返回值。`head` 是一个指向头节点的指针，即遍历开始位置——在我们前面例子中，`fox_list.member` 是 `pos` 中 `list_head` 结构的变量名。这听起来令人迷惑，但是简单实用。下面的代码片断展示了如何重写前面的 `list_for_each()`，来遍历所有 `fox` 节点：

```
struct fox *f;

list_for_each_entry(f, &fox_list, list) {
    /* on each iteration, 'f' points to the next fox structure ... */
}
```

现在看看实际例子吧。它来自 `inotify`——内核文件系统的更新通知机制：

```
static struct inotify_watch *inode_find_handle(struct inode *inode,
                                              struct inotify_handle *ih)
{
    struct inotify_watch *watch;

    list_for_each_entry(watch, &inode->inotify_watches, i_list) {
        if (watch->ih == ih)
            return watch;
    }

    return NULL;
}
```

该函数遍历了 `inode->inotify_watches` 链表中的所有项，每个项的类型都是 `struct inotify_`

watch, list_head 在结构中被命名为 i_list。循环中的每一个遍历, watch 都指向链表的新节点。该函数的目的在于: 在 inode 结构串联起来的 inotify_watches 链表中, 搜寻其 inotify_handle 与所提供的句柄相匹配的 inotify_watch 项。

3. 反向遍历链表

宏 list_for_each_entry_reverse() 的工作和 list_for_each_entry() 类似, 不同点在于它是反向遍历链表的。也就是说, 不再是沿着 next 指针向前遍历, 而是沿着 prev 指针向后遍历。其用法和 list_for_each_entry() 相同:

```
list_for_each_entry_reverse(pos, head, member)
```

很多原因会需要反向遍历链表。其中一个性能原因——如果你知道你要寻找的节点最可能在你搜索的起始点的前面, 那么反向搜索岂不更快。第二个原因是如果顺序很重要, 比如, 如果你使用链表实现堆栈, 那么你需要从尾部向前遍历才能达到先进/先出 (LIFO) 原则。如果你没有确切的反向遍历的原因, 就老实点, 用 list_for_each_entry() 宏吧。

4. 遍历的同时删除

标准的链表遍历方法在你遍历链表的同时要想删除节点时是不行的。因为标准的链表方法建立在你的操作不会改变链表项这一假设上, 所以如果当前项在遍历循环中被删除, 那么接下来的遍历就无法获得 next(或 prev) 指针了。这其实是循环处理中的一个常见范式, 开发人员通过在潜在的删除操作之前存储 next(或者 previous) 指针到一个临时变量中, 以便能执行删除操作。好在 Linux 内核提供了例程处理这种情况:

```
list_for_each_entry_safe(pos, next, head, member)
```

你可以按照 list_for_each_entry() 宏的方式使用上述例程, 只是需要提供 next 指针, next 指针和 pos 是同样的类型。list_for_each_entry_safe() 启用 next 指针来将下一项存进表中, 以使得能安全删除当前项。我们再次看看 inotify 的例子:

```
void inotify_inode_is_dead(struct inode *inode)
{
    struct inotify_watch *watch, *next;

    mutex_lock(&inode->inotify_mutex);
    list_for_each_entry_safe(watch, next, &inode->inotify_watches, i_list) {
        struct inotify_handle *ih = watch->ih;
        mutex_lock(&ih->mutex);
        inotify_remove_watch_locked(ih, watch); /* deletes watch */
        mutex_unlock(&ih->mutex);
    }
    mutex_unlock(&inode->inotify_mutex);
}
```

该函数遍历并删除 inotify_watches 链表中的所有项。如果使用了标准的 list_for_each_entry(), 那么上述代码会造成“使用—在一释放后”的错误, 因为在移向链表中下一项时, 需要访问 watch, 但这时它已经被撤销。

如果你需要在反向遍历链表的同时删除它，那么内核提供了 `list_for_each_entry_safe_reverse()` 宏帮你完成此任务：

```
list_for_each_entry_safe_reverse(pos, n, head, member)
```

上述函数的用法同 `list_for_each_entry_safe()`。

你仍然需要锁定！

`list_for_each_entry()` 的安全版本只能保护你在循环体中从链表中删除数据。如果这时有可能从其他地方并发进行删除，或者有任何其他并发的链表操作，你就需要锁定链表。

请见第9章和第10章中对同步和锁的讨论。

5. 其他链表方法

Linux 提供了很多链表操作方法——几乎是你所能想到的所有访问和操作链表方法，所有这些方法都可在头文件 `<linux/list.h>` 中找到。

6.2 队列

任何操作系统内核都少不了一种编程模型：生产者和消费者。在该模式中，生产者创造数据（比如说需要读取的错误信息或者需要处理的网络包），而消费者则反过来，读取消息和处理包，或者以其他方式消费这些数据。实现该模型的最简单的方式无非是使用队列。生产者将数据推进队列，然后消费者从队列中摘取数据。消费者获取数据的顺序和推入队列的顺序一致。也就是说，第一个进入队列的数据一定是第一个离开队列的。也正是这个原因，队列也称为 FIFO。顾名思义，FIFO 就是先进先出的缩写。图 6-5 是一个标准队列的例子。

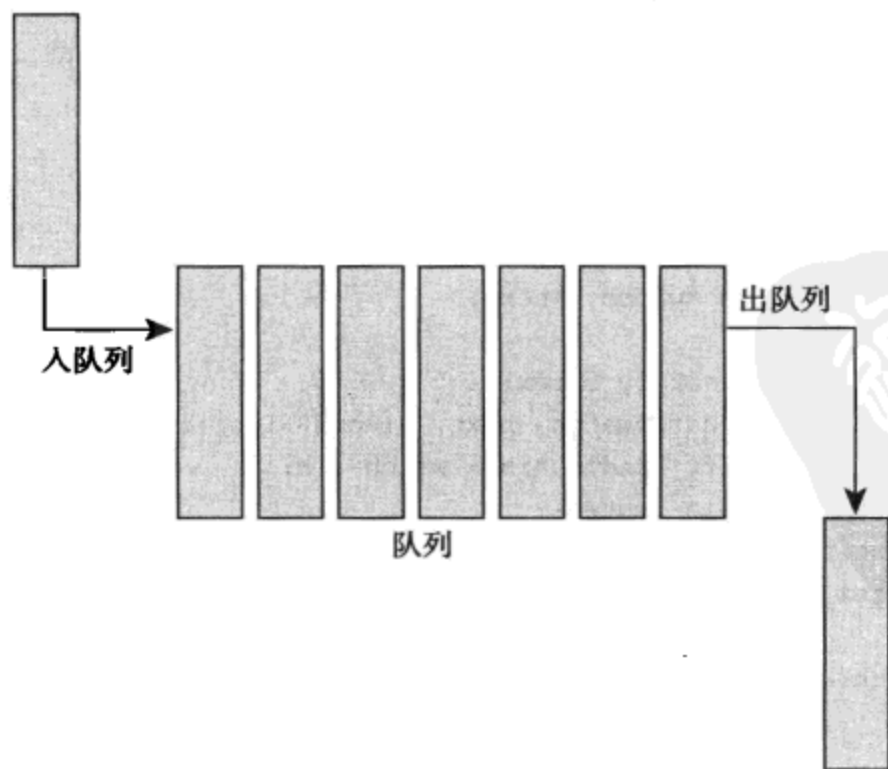


图 6-5 队列 (FIFO)

Linux 内核通用队列实现称为 `kfifo`。它实现在文件 `kernel/kfifo.c` 中，声明在文件 `<linux/kfifo.h>` 中。本节讨论的是自 2.6.33 以后刚更新的 API，使用方法和 2.6.33 前的内核稍有不同，

所以在使用前请仔细检查文件 <linux/kfifo.h>。

6.2.1 kfifo

Linux 的 kfifo 和多数其他队列实现类似，提供了两个主要操作：enqueue（入队列）和 dequeue（出队列）。kfifo 对象维护了两个偏移量：入口偏移和出口偏移。入口偏移是指下一次入队列时的位置，出口偏移是指下一次出队列时的位置。出口偏移总是小于等于入口偏移，否则无意义，因为那样说明要出队列的元素根本还没有入队列。

enqueue 操作拷贝数据到队列中的入口偏移位置。当上述动作完成后，入口偏移随之加上推入的元素数目。dequeue 操作从队列中出口偏移处拷贝数据，当上述动作完成后，出口偏移随之减去摘取的元素数目。当出口偏移等于入口偏移时，说明队列空了：在新数据被推入前，不可再摘取任何数据了。当入口偏移等于队列长度时，说明在队列重置前，不可再有新数据推入队列。

6.2.2 创建队列

使用 kfifo 前，首先必须对它进行定义和初始化。和多数内核对象一样，有动态或者静态方法供你选择。动态方法更为普遍：

```
int kfifo_alloc(struct kfifo *fifo, unsigned int size, gfp_t gfp_mask);
```

该函数创建并且初始化一个大小为 size 的 kfifo。内核使用 gfp_mask 标识分配队列（我们在第 12 章会详细讨论内存分配）。如果成功 kfifo_alloc() 返回 0；错误则返回一个负数错误码。下面便是一个例子：

```
struct kfifo fifo;
int ret;

ret = kfifo_alloc(&kfifo, PAGE_SIZE, GFP_KERNEL);
if (ret)
    return ret;
/* "fifo" 现在代表一个大小为 PAGE_SIZE 的队列... */
```

你要想自己分配缓冲，可以调用：

```
void kfifo_init(struct kfifo *fifo, void *buffer, unsigned int size);
```

该函数创建并初始化一个 kfifo 对象，它将使用由 buffer 指向的 size 字节大小的内存。对于 kfifo_alloc() 和 kfifo_init(), size 必须是 2 的幂。

静态声明 kfifo 更简单，但不大常用：

```
DECLARE_KFIFO(name, size);
INIT_KFIFO(name);
```

上述方法会创建一个名称为 name、大小为 size 的 kfifo 对象。和前面一样，size 必须是 2 的幂。

6.2.3 推入队列数据

当你的 kfifo 对象创建和初始化后，推入数据到队列需要通过 kfifo_in() 方法完成：

```
unsigned int kfifo_in(struct kfifo *fifo, const void *from, unsigned int len);
```

该函数把 from 指针所指的 len 字节数据拷贝到 fifo 所指的队列中，如果成功，则返回推入数据的字节大小。如果队列中的空闲字节小于 len，则该函数值最多可拷贝队列可用空间那么多的数据，这样的话，返回值可能小于 len，甚至会返回 0，这时意味着没有任何数据被推入。

6.2.4 摘取队列数据

推入数据使用函数 kfifo_in()，摘取数据则需要通过函数 kfifo_out() 完成：

```
unsigned int kfifo_out(struct kfifo *fifo, void *to, unsigned int len);
```

该函数从 fifo 所指向的队列中拷贝出长度为 len 字节的数据到 to 所指的缓冲中。如果成功，该函数则返回拷贝的数据长度。如果队列中数据大小小于 len，则该函数拷贝出的数据必然小于需要的数据大小。

当数据被摘取后，数据就不再存在于队列之中。这是队列操作的常用方式。不过如果仅仅想“偷窥”队列中的数据，而不真想删除它，你可以使用 kfifo_out_peek() 方法：

```
unsigned int kfifo_out_peek(struct kfifo *fifo, void *to, unsigned int len,  
                           unsigned offset);
```

该函数和 kfifo_out() 类似，但出口偏移不增加，而且摘取的数据仍然可被下次 kfifo_out 获得。参数 offset 指向队列中的索引位置，如果该参数为 0，则读队列头，这和 kfifo_out() 无异。

6.2.5 获取队列长度

若想获得用于存储 kfifo 队列的空间的总体大小（以字节为单位），可调用方法 kfifo_size()：

```
static inline unsigned int kfifo_size(struct kfifo *fifo);
```

另一个内核命名不佳的例子来了——kfifo_len() 方法返回 kfifo 队列中已推入的数据大小：

```
static inline unsigned int kfifo_len(struct kfifo *fifo);
```

如果想得到 kfifo 队列中还有多少可用空间，则要调用方法：

```
static inline unsigned int kfifo_avail(struct kfifo *fifo);
```

最后两个方法是 kfifo_is_empty() 和 kfifo_is_full()。如果给定的 kfifo 分别是空或者满，它们返回非 0 值。如果返回 0，则相反。

```
static inline int kfifo_is_empty(struct kfifo *fifo);  
static inline int kfifo_is_full(struct kfifo *fifo);
```

6.2.6 重置和撤销队列

如果重置 kfifo，意味着抛弃所有队列中的内容，调用 kfifo_reset()：

```
static inline void kfifo_reset(struct kfifo *fifo);
```

撤销一个使用 kfifo_alloc() 分配的队列，调用 kfifo_free()：

```
void kfifo_free(struct kfifo *fifo);
```

如果你是使用 `kfifo_init()` 方法创建的队列，那么你需要负责释放相关的缓冲。具体方法取决于你是如何创建它的。去看看第 12 章关于动态分配和释放内存的讨论吧。

6.2.7 队列使用举例

使用上述接口，我们看一个 `kfifo` 的具体用例。假定我们创建了一个由 `fifo` 指向的 8KB 大小的 `kfifo`。我们就可以推入数据到队列。这个例子中，我们推入简单的整型数。在你自己的代码中，可以推入更复杂的任务相关数据。这里使用整数，我们看看 `kfifo` 如何工作：

```
unsigned int i;

/* 将 [0,32) 压入到名为 'fifo' 的 kfifo 中 */
for (i = 0; i < 32; i++)
    kfifo_in(fifo, &i, sizeof(i));
```

名为 `fifo` 的 `kfifo` 现在包含了 0 到 31 的整数，我们查看一下队列的第一个元素是不是 0：

```
unsigned int val;
int ret;

ret = kfifo_out_peek(fifo, &val, sizeof(val), 0);
if (ret != sizeof(val))
    return -EINVAL;
printk(KERN_INFO "%u\n", val); /* 应该输出 0 */
```

摘取并打印 `kfifo` 中的所有元素，我们可以调用 `kfifo_out()`：

```
/* 当队列中还有数据时 */
while (kfifo_peek(fifo)) {
    unsigned int val;
    int ret;

    /* ... read it, one integer at a time */
    ret = kfifo_out(fifo, &val, sizeof(val));
    if (ret != sizeof(val))
        return -EINVAL;

    printk(KERN_INFO "%u\n", val);
}
```

0 到 31 的整数将一一按序打印出来（如果需要逆序打印，即从 31 到 0，那么我们应该使用堆栈而不是队列）。

6.3 映射

一个映射，也常称为关联数组，其实是一个由唯一键组成的集合，而每个键必然关联一个特定的值。这种键到值的关联关系称为映射。映射要至少支持三个操作：

- * Add (key, value)
- * Remove (key)
- * value = Lookup (key)

虽然散列表是一种映射，但并非所有的映射都需要通过散列表实现。除了使用散列表外，映射也可以通过自平衡二叉搜索树存储数据。虽然散列表能提供更好的平均的渐近复杂度（请看本章后面关于算法复杂度的讨论），但是二叉搜索树在最坏情况下能有更好的表现（即对数复杂性相比线性复杂性）。二叉搜索树同时满足顺序保证，这将给用户的按序遍历带来很好的性能。二叉搜索树的最后一个优势是它不需要散列函数，需要的键类型只要可以定义 $\leq$ 操作算子便可以。

虽然键到值的映射属于一个通用说法，但是更多时候特指使用二叉树而非散列表实现的关联数组。比如，C++ 的 STL 容器 `std::map` 便是采用自平衡二叉搜索树（或者类似的数据结构）实现的，它能提供按序遍历的能力。

Linux 内核提供了简单、有效的映射数据结构。但是它并非一个通用的映射。因为它的目标是：映射一个唯一的标识数（UID）到一个指针。除了提供三个标准的映射操作外，Linux 还在 `add` 操作基础上实现了 `allocate` 操作。这个 `allocate` 操作不但向 `map` 中加入了键值对，而且还可产生 UID。

`idr` 数据结构用于映射用户空间的 UID，比如将 `inotify watch` 的描述符或者 POSIX 的定时器 ID 映射到内核中相关联的数据结构上，如 `inotify_watch` 或者 `k_itimer` 结构体。其命名仍然沿袭了内核中有些含混不清的命名体系，这个映射被命名为 `idr`。

6.3.1 初始化一个 `idr`

建立一个 `idr` 很简单，首先你需要静态定义或者动态分配一个 `idr` 数据结构。然后调用 `idr_init()`：

```
void idr_init(struct idr *idp);
```

比如：

```
struct idr id_huh; /* 静态定义 idr 结构 */
idr_init(&id_huh); /* 初始化 idr 结构 */
```

6.3.2 分配一个新的 UID

一旦建立了 `idr`，就可以分配新的 UID 了，这个过程分两步完成。第一步，告诉 `idr` 你需要分配新的 UID，允许其在必要时调整后备树的大小。然后，第二步才是真正请求新的 UID。之所以需要这两个组合动作是因为要允许调整初始大小——这中间涉及在无锁情况下分配内存的场景。我们在第 12 章将讨论内存分配，在第 9 章和第 10 章讨论加锁问题。现在我们先别管如何处理上锁问题，重点看看如何使用 `idr`。

第一个调整后备树大小的方法是 `idr_pre_get()`：

```
int idr_pre_get(struct idr *idp, gfp_t gfp_mask);
```

该函数将在需要进行 UID 的分配工作：调整由 `idp` 指向的 `idr` 的大小。如果真的需要调整大小，则内存分配例程使用 `gfp` 标识：`gfp_mask`（`gfp` 标识将在第 12 章讨论），你不需要对并发访问该方法进行同步保护。和内核中其他函数的做法相反，`idr_pre_get()` 成功时返回 1，失败时返

回 0——这点一定要注意。

第二个函数，实际执行获取新的 UID，并且将其加到 idr 的方法是 `idr_get_new()`：

```
int idr_get_new(struct idr *idp, void *ptr, int *id);
```

该方法使用 `idp` 所指向的 `idr` 去分配一个新的 UID，并且将其关联到指针 `ptr` 上。成功时，该方法返回 0，并且将新的 UID 存于 `id`。错误时，返回非 0 的错误码，错误码是 `-EAGAIN`，说明你需要（再次）调用 `idr_pre_get()`；如果 `idr` 已满，错误码是 `-ENOSPC`。

看一个完整例子吧：

```
int id;

do {
    if (!idr_pre_get(&idr_huh, GFP_KERNEL))
        return -ENOSPC;
    ret = idr_get_new(&idr_huh, ptr, &id);
} while (ret == -EAGAIN);
```

如果成功，上述代码片段将获得一个新的 UID，它被存储在整型变量 `id` 中，而且将 UID 映射到 `ptr`（我们没有在代码片段中定义它）

函数 `idr_get_new_above()` 使得调用者可指定一个最小的 UID 返回值：

```
int idr_get_new_above(struct idr *idp, void *ptr, int starting_id, int *id);
```

该函数的作用和 `idr_get_new()` 相同，除了它确保新的 UID 大于或等于 `starting_id` 外。使用这个变种方法允许 `idr` 的使用者确保 UID 不会被重用，允许其值不但在当前分配的 ID 中唯一，而且还保证在系统的整个运行期间唯一。下面的代码片段和前例中的类似，不过我们明确要求增加 UID 的值：

```
int id;

do {
    if (!idr_pre_get(&idr_huh, GFP_KERNEL))
        return -ENOSPC;
    ret = idr_get_new_above(&idr_huh, ptr, next_id, &id);
} while (ret == -EAGAIN);

if (!ret)
    next_id = id + 1;
```

6.3.3 查找 UID

当我们在一个 `idr` 中已经分配了一些 UID 时，我们自然就需要查找它们：调用者要给出 UID，`idr` 将返回对应的指针。查找步骤显然要比分配一个新 UID 要来的简单，仅需使用 `idr_find()` 方法即可：

```
void *idr_find(struct idr *idp, int id);
```

该函数如果调用成功，则返回 id 关联的指针；如果错误，则返回空指针。注意，如果你使用 `idr_get_new()` 或者 `idr_get_new_above()` 将空指针映射给 UID，那么该函数在成功时也返回 NULL。这样你就无法区分是成功还是失败，所以，最好不要将 UID 映射到空指针上。

这个函数的使用比较简单：

```
struct my_struct *ptr = idr_find(&idr_huh, id);
if (!ptr)
    return -EINVAL; /* 错误 */
```

6.3.4 删除 UID

从 idr 中删除 UID 使用方法 `idr_remove()`：

```
void idr_remove(struct idr *idp, int id);
```

如果 `idr_remove()` 成功，则将 id 关联的指针一起从映射中删除。遗憾的是，`idr_remove()` 并没有办法提示任何错误（比如，如果 id 不在 idp 中）。

6.3.5 撤销 idr

撤销一个 idr 的操作很简单，调用 `idr_destroy()` 函数即可：

```
void idr_destroy(struct idr *idp);
```

如果该方法成功，则只释放 idr 中未使用的内存。它并不释放当前分配给 UID 使用的任何内存。通常，内核代码不会撤销 idr，除非关闭或者卸载，而且只有在没有其他用户（也就没有更多的 UID）时才能删除，但是你可以调用 `idr_remove_all()` 方法强制删除所有的 UID：

```
void idr_remove_all(struct idr *idp);
```

你应该首先对 idp 指向的 idr 调用 `idr_remove_all()`，然后再调用 `idr_destroy()`，这样就能使 idr 占用的内存都被释放。

6.4 二叉树

树结构是一个能提供分层的树型数据结构的特定数据结构。在数学意义上，树是一个无环的、连接的有向图，其中任何一个顶点（在树里叫节点）具有 0 个或者多个出边以及 0 个或者 1 个入边。一个二叉树是每个节点最多只有两个出边的树——也就是，一个树，其节点具有 0 个、1 个或者 2 个子节点。请见图 6-6 所示的简单二叉树。

6.4.1 二叉搜索树

一个二叉搜索树（通常简称为 BST）是一个节点有序的

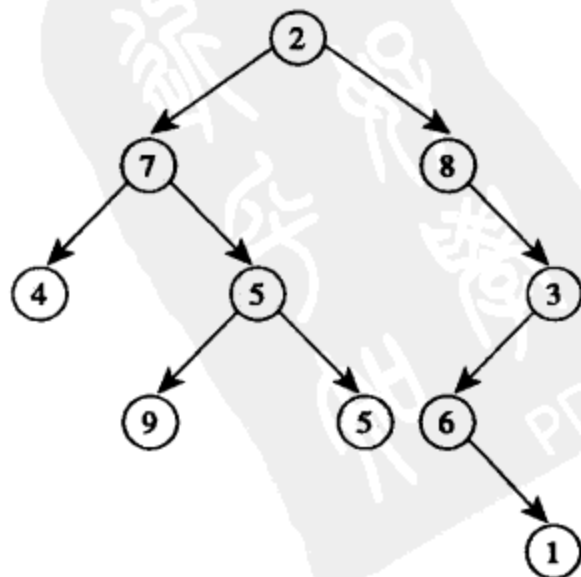


图 6-6 二叉树

二叉树，其顺序通常遵循下列法则：

- 根的左分支节点值都小于根节点值。
- 右分支节点值都大于根节点值。
- 所有的子树也都是二叉搜索树。

因此，一个二叉搜索树所有节点必然都有序，且左子节点小于其父节点值，而右子节点大于其父节点值的二叉树。所以，在树中搜索一个给定值或者按序遍历树都相当快捷（算法分别是对数和线性的）。见图 6-7 给出的简单二叉搜索树。

6.4.2 自平衡二叉搜索树

一个节点的深度是指从其根节点起，到达它一共需经过的父节点数目。处于树底层的节点（再也没有子节点）称为叶子节点。一个树的高度是指树中的处于最底层节点的深度。一个平衡二叉搜索树是一个所有叶子节点深度差不超过 1 的二叉搜索树（见图 6-8）。一个自平衡二叉搜索树是指其操作都试图维持（半）平衡的二叉搜索树。

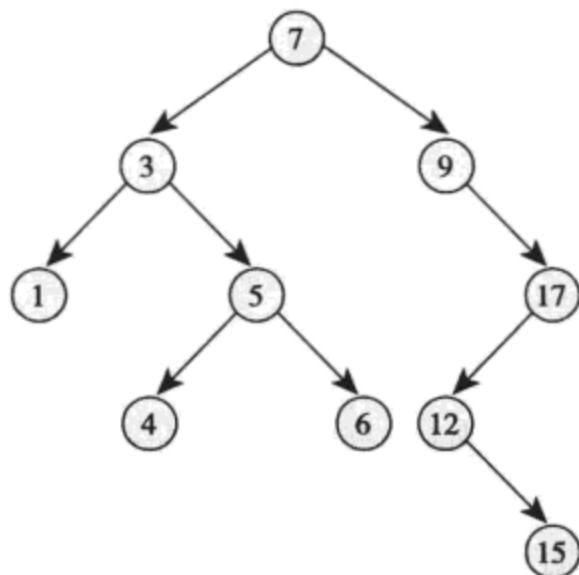


图 6-7 二叉搜索树 (BST)

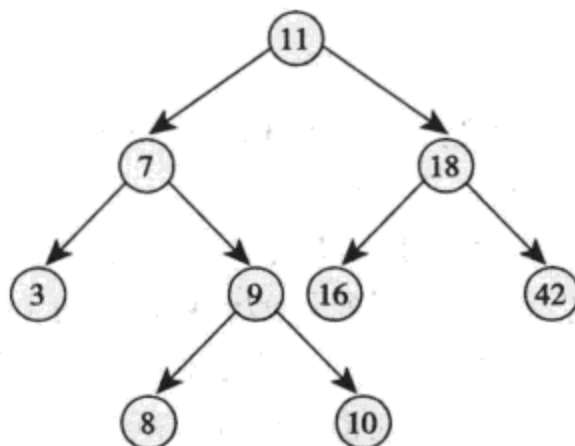


图 6-8 平衡二叉搜索树

1. 红黑树

红黑树是一种自平衡二叉搜索树。Linux 主要的平衡二叉树数据结构就是红黑树。红黑树具有特殊的着色属性，或红色或黑色。红黑树因遵循下面六个属性，所以能维持半平衡结构：

- (1) 所有的节点要么着红色，要么着黑色。
- (2) 叶子节点都是黑色。
- (3) 叶子节点不包含数据。
- (4) 所有非叶子节点都有两个子节点。
- (5) 如果一个节点是红色，则它的子节点都是黑色。

(6) 在一个节点到其叶子节点的路径中，如果总是包含同样数目的黑色节点，则该路径相比其他路径是最短的。

上述条件，保证了最深的叶子节点的深度不会大于两倍的最浅叶子节点的深度。所以，红黑树总是半平衡的。为什么它具有如此神奇的特点呢？首先，第五个属性，一个红色节点不能是其

他红色节点的子节点或者父节点。而第六个属性保证了，从树的任何节点到其叶子节点的路径都具有相同数目的黑色节点，树里的最长路径则是红黑交替节点路径，所以最短路径必然是具有相同数量黑色节点的——只包含黑色节点的路径。于是从根节点到叶子节点的最长路径不会超过最短路径的两倍。

如果插入和删除操作可以遵循上述六个要求，那这个树会始终保持是一个半平衡树。看起来也许有些奇怪，为什么插入和删除动作都需要服从这些特别的约束，为什么不能用一些简单的规则去维持平衡树呢？其实，实践证明这些规则遵循起来还是相对简单（虽然实现复杂）的。而且在保证半平衡树前提下，这些插入和删除动作并不会增加额外负担。

至于如何让插入和删除动作都能遵循这些规则，已经超出了本书范围。相比简单的规则，实现起来可要复杂得多。不过任何好点的大学数据结构教科书上都应该有完整的讲述。

2. rbtree

Linux 实现的红黑树称为 rbtree。其定义在文件 lib/rbtree.c 中，声明在文件 <linux/rbtree.h> 中。除了一定的优化外，Linux 的 rbtree 类似于前面所描述的经典红黑树，即保持了平衡性，所以插入效率和树中节点数目呈对数关系。

rbtree 的根节点由数据结构 rb_root 描述。创建一个红黑树，我们要分配一个新的 rb_root 结构，并且需要初始化为特殊值 RB_ROOT：

```
struct rb_root root = RB_ROOT;
```

树里的其他节点由结构 rb_node 描述。给定一个 rb_node，我们可以通过跟踪同名节点指针来找到它的左右子节点。

rbtree 的实现并没有提供搜索和插入例程，这些例程希望由 rbtree 的用户自己定义。这是因为 C 语言不大容易进行泛型编程，同时 Linux 内核开发者们相信最有效的搜索和插入方法需要每个用户自己去实现。你可以使用 rbtree 提供的辅助函数，但你自己要实现比较操作算子。

搜索操作和插入操作最好的范例就是展示一个实际场景：我们先来看搜索，下面的函数实现了在页高速缓存中搜索一个文件区（由一个 i 节点和一个偏移量共同描述）。每个 i 节点都有自己的 rbtree，以关联在文件中的页偏移。该函数将搜索给定 i 节点的 rbtree，以寻找匹配的偏移值：

```
struct page * rb_search_page_cache(struct inode *inode,
                                   unsigned long offset)
{
    struct rb_node *n = inode->i_rb_page_cache.rb_node;

    while (n) {
        struct page *page = rb_entry(n, struct page, rb_page_cache);
        if (offset < page->offset)
            n = n->rb_left;
        else if (offset > page->offset)
            n = n->rb_right;
        else
            return page;
    }
    return NULL;
}
```

这个例子中，在 while 循环中遍历了整个 rbtree。offset 将决定是向左或是向右遍历。if 和 else 条件实际上实现了 rbtree 的比较方法，从而确保了树的有序性。如果循环中找到了一个匹配 offset 的节点，则搜索完成，并返回对应的 page 结构。如果循环查找了全树也没有找到一个匹配项，说明在树中不存在匹配项，则函数返回 NULL。

插入操作要相对复杂一些，因为必须实现搜索和插入逻辑。下面并非一个了不起的函数，但可以作为你实现自己的插入操作的一个指导：

```
struct page * rb_insert_page_cache(struct inode *inode,
                                   unsigned long offset,
                                   struct rb_node *node)
{
    struct rb_node **p = &inode->i_rb_page_cache.rb_node;
    struct rb_node *parent = NULL;
    struct page *page;

    while (*p) {
        parent = *p;
        page = rb_entry(parent, struct page, rb_page_cache);

        if (offset < page->offset)
            p = &(*p)->rb_left;
        else if (offset > page->offset)
            p = &(*p)->rb_right;
        else
            return page;
    }

    rb_link_node(node, parent, p);
    rb_insert_color(node, &inode->i_rb_page_cache);

    return NULL;
}
```

和搜索操作一样，while 循环需要遍历整个树，也是根据 offset 选择遍历方向。但是和搜索不同的是，该函数希望找不到匹配的 offset，因为它想要找的是新 offset 要插入的叶子节点。当插入点找到后，调用 rb_link_node() 在给定位位置插入新节点。接着调用 rb_insert_color() 方法执行复杂的再平衡动作。如果页被加入到页高速缓存中，则返回 NULL。如果页已经在高速缓存中了，则返回这个已存在的页结构地址。

6.5 数据结构以及选择

我们已经详细讨论了 Linux 中最重要的四种数据结构：链表、队列、映射和红黑树。在本节中，我们将教你如何在代码中具体选择使用哪种数据结构。

如果你对数据集合的主要操作是遍历数据，就使用链表。事实上没有数据结构可以提供比线性算法复杂度更好的算法遍历元素，所以你应该用最简单的数据结构完成简单工作。另外，当性能并非首要考虑因素时，或者当你需要存储相对较少的数据项时，或者当你需要和内核中其他使

用链表的代码交互时，也该优先选择链表。

如果你的代码符合生产者/消费者模式，则使用队列，特别是如果你想（或者可以）要一个定长缓冲。队列会使得添加和删除项的工作简单有效。同时队列也提供了先入先出（FIFO）语义，而这也正是生产者/消费者用例的普遍需求。另一方面，如果你需要存储一个大小不明的（可能很多项）的数据集合，那么链表可能更合适，因为你可以动态添加任何数量的数据项。

如果你需要映射一个 UID 到一个对象，就使用映射。映射结构使得映射工作简单有效，而且映射可以帮你维护和分配 UID。Linux 的映射接口是针对 UID 到指针的映射，它并不适合其他场景。但是如果你在处理发给用户空间的描述符，就考虑一下映射吧。

如果你需要存储大量数据，并且检索迅速，那么红黑树最好。红黑树可确保搜索时间复杂度是对数关系，同时也能保证按序遍历时间复杂度是线性关系。虽然它比其他数据结构复杂一些，但其内存开销情况并不是太糟。但是如果你没有执行太多次时间紧迫的查找操作，则红黑树可能不是最好选择。这种情况最好使用链表。

要是上述数据结构都不能满足你的需要，内核还实现了一些较少使用的数据结构，也许它们能帮你，比如基树（trie 类型）和位图。只有当寻遍所有内核提供的数据结构都不能满足时，你才需要自己设计数据结构。经常在独立的源文件中实现的一种常见数据结构是散列表。因为散列表无非是一些“桶”和一个散列函数，而且这个散列函数是针对每个用例的，因此用非泛型编程语言（如 C 语言）实现内核范围内的统一散列表，其实这并没有什么价值。

6.6 算法复杂度

在计算机科学和相关的学科中，很有必要将算法的复杂度（或伸缩度）量化地表示出来。虽然存在各种各样表示伸缩度的方法，但最常用的技术还是研究算法的渐近行为（asymptotic behavior）。渐近行为是指当算法的输入变得非常大或接近于无限大时算法的行为。渐近行为充分显示了当一个算法的输入逐渐变大时，该算法的伸缩度如何。研究算法的伸缩度（当输入增大时算法执行的变化）可以帮助我们以特定基准抽象出算法模型，从而更好地理解算法的行为。

6.6.1 算法

算法就是一系列的指令，它可能有一个或多个输入，最后产生一个结果或输出。比如计算一个房间中人数的步骤就是一个算法，它的输入是人，计数结果是输出。在 Linux 内核中，页换出和进程调度都是算法的例子。从数学角度讲，一个算法好比一个函数（或至少我们可将它抽象为一个函数）。比如，我们称人数统计算法为 f ，要统计的人数为 x ，可以写成下面形式：

$$y = f(x) \quad \text{人数统计的函数}$$

这里 y 是统计 x 个人所需的时间。

6.6.2 大 o 符号

一种很有用的渐近表示法就是上限——它是一个函数，其值自从一个起始点之后总是超过我们所研究的函数的值，也就是说上限增长等于或者快于我们研究的函数。一个特殊符号，大 o 符

号用来描述这种增长率。函数 $f(x)$ 可写作 $O(g(x))$, 读为“ f 是 g 的大 o ”。数学定义形式为：

如果 $f(x)$ 是 $O(g(x))$, 那么

$\exists c, x'$ 满足 $f(x) \leq c \cdot g(x), \forall x > x'$

换成自然语言就是, 完成 $f(x)$ 的时间总是短于或等于完成 $g(x)$ 的时间和任意常量 (至少, 只要输入的 x 值大于某个初始值 x') 的乘积。

从根本上讲, 我们需要寻找一个函数, 它的行为和我们的算法一样差或更差。这样一来我们就可以通过给该函数送入非常大的输入, 然后观察该函数的结果, 从而了解我们算法的执行上限。

6.6.3 大 θ 符号

当大多数人谈论大 o 符号时, 更准确地讲他们谈论的更接近 Donald Knuth 所描述的大 θ 符号。从技术角度讲, 大 o 符号适合描述上限, 比如 7 是 6 的上限, 同样道理, 9、12 和 65 也都是 6 的上限。但在后来大多数人讨论函数增长率时, 更多说的是最小上限, 或一个抽象出具有上限和下限^①的函数。算法分析领域之父, Knuth 教授, 将其描述为大 θ 符号, 并给出了下面的定义:

如果 $f(x)$ 是 $g(x)$ 的大 θ , 那么 $g(x)$ 既是 $f(x)$ 的上限也是 $f(x)$ 的下限。

那么, 我们也可以说 $f(x)$ 是 $g(x)$ 级 (order)。级或大 θ , 是理解内核中算法的最重要的数学工具之一。

所以, 当人们谈到大 o 符号时, 他们往往是在谈论大 θ 。当然你不用为此担心, 除非你想讨 Knuth 教授欢心。

6.6.4 时间复杂度

比如, 再次考虑计算房间里的人数, 假设你一秒钟数一个人, 那么如果有 7 个人在房间里, 你需要花 7 秒钟数它们。显然如果有 n 个人, 需要花 n 秒来数它们。我们称该算法复杂度为 $O(n)$ 。如果任务是在房间里所有人面前跳舞呢? 因为不管房间里有 5 个人还是有 5 000 个人, 跳舞花费的时间都是相同的, 所以该任务的复杂度为 $O(1)$ 。表 6-1 给出了常见的复杂度。

表 6-1 时间复杂度表

$O(g(x))$	名 称
1	恒量 (理想的伸缩度)
$\log n$	对数的
n	线性的
n^2	平方的
n^3	立方的
2^n	指数的
$n!$	阶乘

① 如果你好奇, 下限使用大 ω 符号建模, 其定义除了 $g(x)$ 总是小于或等于而不是大于或等于 $f(x)$ 外, 和大 o 相同。大 ω 表示没有大 o 表示有用, 因为发现函数甚至还小于你的函数, 这就对函数的行为几乎没有指示性。

让房间里的所有人相互介绍的复杂度是多少呢？有什么函数抽象这种算法呢？如果介绍一个人需要花费 30 秒，那么相互介绍 10 个人花多久呢？介绍 100 个人又需要花多久呢？理解一个算法在提高工作负载时的表现，是为给定工作选择最好算法的关键。

显然，应避免使用复杂度为 $O(n!)$ 或 $O(2^n)$ 的算法，另外，用复杂度为 $O(1)$ 的函数代替复杂度为 $O(n)$ 的函数通常都会提高执行性能。但是情况并非总是如此，不能仅仅依靠算法复杂度（大 o 符号）来判断哪种算法在实际使用中性能更高。回忆一下，指定的 $O(g(x))$ ，有一个恒量 c 和 $g(x)$ 相乘，所以有可能复杂度为 $O(1)$ 的算法需要花费 3 个小时才能完成任务，而且无论输入多大，总是要花 3 个小时。这样的话很可能要比复杂度为 $O(n)$ 、但输入很少的算法费时还长。因此我们在比较算法性能时，还需要考虑输入规模。

我们不赞成使用复杂的算法，但是时刻要注意算法的负载和典型输入集合大小的关系。不要为了你根本不需要支持的伸缩度要求，盲目地去优化算法。

6.7 小结

本章我们讨论了许多 Linux 内核开发者们用于实现从进程调度到设备驱动等内核代码的通用数据结构。你会随着学习的深入，慢慢发现这些数据结构的妙用。你写自己的内核代码时，记住总是应该重用已经存在的内核基础设施，别去重复造轮子！

我们也介绍了算法复杂度以及测量和标识算法复杂度的工具，其中最值得注意的是大 o 。贯穿本书，以及 Linux 内核，大 o 都是我们评价算法和内核组件在多用户、处理器、进程、网络连接，以及其他环境下伸缩度的重要指标。



第 ⑦ 章

中断和中断处理

任何操作系统内核的核心任务，都包含有对连接到计算机上的硬件设备进行有效管理，如硬盘、蓝光碟机、键盘、鼠标、3D 处理器，以及无线电等。而想要管理这些设备，首先要能和它们互通音信才行。众所周知，处理器的速度跟外围硬件设备的速度往往不在一个数量级上，因此，如果内核采取让处理器向硬件发出一个请求，然后专门等待回应的办法，显然差强人意。既然硬件的响应这么慢，那么内核就应该在此期间处理其他事务，等到硬件真正完成了请求的操作之后，再回过头来对它进行处理。

那么到底如何让处理器和这些外部设备能协同工作，且不会降低机器的整体性能呢？轮询（polling）可能会是一种解决办法。它可以让内核定期对设备的状态进行查询，然后做出相应的处理。不过这种方法很可能会让内核做不少无用功，因为无论硬件设备是正在忙碌着完成任务还是已经大功告成，轮询总会周期性地重复执行。更好的办法是由我们来提供一种机制，让硬件在需要的时候再向内核发出信号^①。这就是中断机制。在本章中，我们将先讨论中断，进而讨论内核如何使用所谓的中断处理函数处理对应的中断。

7.1 中断

中断使得硬件得以发出通知给处理器。例如，在你敲击键盘的时候，键盘控制器（控制键盘的硬件设备）会发送一个中断，通知操作系统有键按下。中断本质上是一种特殊的电信号，由硬件设备发向处理器。处理器接收到中断后，会马上向操作系统反映此信号的到来，然后就由操作系统负责处理这些新到来的数据。硬件设备生成中断的时候并不考虑与处理器的时钟同步——换句话说就是中断随时可以产生。因此，内核随时可能因为新到来的中断而被打断。

从物理学的角度看，中断是一种电信号，由硬件设备生成，并直接送入中断控制器的输入引脚中——中断控制器是个简单的电子芯片，其作用是将多路中断管线，采用复用技术只通过一个和处理器相连接的管线与处理器通信。当接收到一个中断后，中断控制器会给处理器发送一个电信号。处理器一经检测到此信号，便中断自己的当前工作转而处理中断。此后，处理器会通知操作系统已经产生中断，这样，操作系统就可以对这个中断进行适当地处理了。

不同的设备对应的中断不同，而每个中断都通过一个唯一的数字标志。因此，来自键盘的中断就有别于来自硬盘的中断，从而使得操作系统能够对中断进行区分，并知道哪个硬件设备产生了哪个中断。这样，操作系统才能给不同的中断提供对应的中断处理程序。

① 变内核主动为硬件主动。——译者注

这些中断值通常被称为中断请求（IRQ）线。每个 IRQ 线都会被关联一个数值量——例如，在经典的 PC 机上，IRQ 0 是时钟中断，而 IRQ 1 是键盘中断。但并非所有的中断号都是这样严格定义的。例如，对于连接在 PCI 总线上的设备而言，中断是动态分配的。而且其他非 PC 的体系结构也具有动态分配可用中断的特性。重点在于特定的中断总是与特定的设备相关联，并且内核要知道这些信息。实际上，硬件发出中断是为了引起内核的关注：嗨，我有新的按键等待处理呢，读取并处理这些调皮鬼吧！

异常

在操作系统中，讨论中断就不能不提及异常。异常与中断不同，它在产生时必须考虑与处理器时钟同步。实际上，异常也常常称为同步中断。在处理器执行到由于编程失误而导致的错误指令（如被 0 除）的时候，或者是在执行期间出现特殊情况（如缺页），必须靠内核来处理的时候，处理器就会产生一个异常。因为许多处理器体系结构处理异常与处理中断的方式类似，因此，内核对它们的处理也很类似。本章对中断（由硬件产生的异步中断）的讨论，大部分也适合于异常（由处理器本身产生的同步中断）。

你已经熟悉一种异常：在第 6 章中你已看到，在 x86 体系结构上如何通过软中断实现系统调用，那就是陷入内核，然后引起一种特殊的异常——系统调用处理程序异常。你会看到，中断的工作方式与之类似，其差异只在于中断是由硬件而不是软件引起的。

7.2 中断处理程序

在响应一个特定中断的时候，内核会执行一个函数，该函数叫做中断处理程序（interrupt handler）或中断服务例程（interrupt service routine, ISR）。产生中断的每个设备都有一个^①相应的中断处理程序。例如，由一个函数专门处理来自系统时钟的中断，而另外一个函数专门处理由键盘产生的中断。一个设备的中断处理程序是它设备驱动程序（driver）的一部分——设备驱动程序是用于对设备进行管理的内核代码。

在 Linux 中，中断处理程序就是普普通通的 C 函数。只不过这些函数必须按照特定的类型声明，以便内核能够以标准的方式传递处理程序的信息，在其他方面，它们与一般的函数别无二致。中断处理程序与其他内核函数的真正区别在于，中断处理程序是被内核调用来响应中断的，而它们运行于我们称之为中断上下文的特殊上下文中（关于中断上下文，我们将在后面讨论）。需要指出的是，中断上下文偶尔也称作原子上下文，因为正如我们看到的，该上下文中的执行代码不可阻塞。不过在本书中我们使用中断上下文这个称谓。

中断可能随时发生，因此中断处理程序也就随时可能执行。所以必须保证中断处理程序能够快速执行，这样才能保证尽可能快地恢复中断代码的执行。因此，尽管对硬件而言，操作系统能迅速对其中断进行服务非常重要；当然对系统的其他部分而言，让中断处理程序在尽可能短的时间内完成运行也同样重要。

① 中断处理程序通常不是和特定设备关联，而是和特定中断关联的，也就是说，如果一个设备可以产生多种不同的中断，那么该设备就可以对应多个中断处理程序，相应的，该设备的驱动程序也就需要准备多个这样的函数。

最起码的，中断处理程序要负责通知硬件设备中断已被接收：嗨，硬件，我听到你了，现在回去工作吧！但是中断处理程序往往还要完成大量其他的工具。例如，我们可以考虑一下网络设备的中断处理程序面临的挑战。该处理程序除了要对硬件应答，还要把来自硬件的网络数据包拷贝到内存，对其进行处理后再交给合适的协议栈或应用程序。显而易见，这种工作量不会太小，尤其对于如今的千兆比特和万兆比特以太网卡而言。

7.3 上半部与下半部的对比

又想中断处理程序运行得快，又想中断处理程序完成的工作量多，这两个目的显然有所抵触。鉴于两个目的之间存在此消彼长的矛盾关系，所以我们一般把中断处理切为两个部分或两半。中断处理程序是上半部（top half）——接收到一个中断，它就立即开始执行，但只做有严格时限的工作，例如对接收的中断进行应答或复位硬件，这些工作都是在所有中断被禁止的情况下完成的。能够被允许稍后完成的工作会推迟到下半部（bottom half）去。此后，在合适的时机，下半部会被开中断执行。Linux 提供了实现下半部的各种机制，第 8 章会讨论这些机制。

让我们考察一下上半部和下半部分割的例子，还是以我们的老朋友——网卡作为实例。当网卡接收来自网络的数据包时，需要通知内核数据包到了。网卡需要立即完成这件事，从而优化网络的吞吐量和传输周期，以避免超时。因此，网卡立即发出中断：嗨，内核，我这里有最新数据包了。内核通过执行网卡已注册的中断处理程序来做出应答。

中断开始执行，通知硬件，拷贝最新的网络数据包到内存，然后读取网卡更多的数据包。这些都是重要、紧迫而又与硬件相关的工作。内核通常需要快速的拷贝网络数据包到系统内存，因为网卡上接收网络数据包的缓存大小固定，而且相比系统内存也要小得多。所以上述拷贝动作一旦被延迟，必然造成缓存溢出——进入的网络包占满了网卡的缓存，后续入包只能被丢弃。当网络数据包被拷贝到系统内存后，中断的任务算是完成了，这时它将控制权交还给系统被中断前原先运行的程序。处理和操作数据包的其他工作在随后的下半部中进行。本章，我们考察上半部；第 8 章，我们关注下半部。

7.4 注册中断处理程序

中断处理程序是管理硬件的驱动程序的组成部分。每一设备都有相关的驱动程序，如果设备使用中断（大部分设备如此），那么相应的驱动程序就注册一个中断处理程序。

驱动程序可以通过 `request_irq()` 函数注册一个中断处理程序（它被声明在文件 `<linux/interrupt.h>` 中），并且激活给定的中断线，以处理中断：

```
/* request_irq: 分配一条给定的中断线 */
int request_irq(unsigned int irq,
               irq_handler_t handler,
               unsigned long flags,
               const char *name,
               void *dev)
```


第一个参数 `irq` 表示要分配的中断号。对某些设备，如传统 PC 设备上的系统时钟或键盘，这个值通常是预先确定的。而对于大多数其他设备来说，这个值要么是通过探测获取，要么可以通过编程动态确定。

第二个参数 `handler` 是一个指针，指向处理这个中断的实际中断处理程序。只要操作系统一接收到中断，该函数就被调用。

```
typedef irqreturn_t (*irq_handler_t)(int, void *);
```

注意 `handler` 函数的原型，它接受两个参数，并有一个类型为 `irqreturn_t` 的返回值。我们将在本章随后的部分讨论这个函数。

7.4.1 中断处理程序标志

第三个参数 `flags` 可以为 0，也可能是下列一个或多个标志的位掩码。其定义在文件 `<linux/interrupt.h>`。在这些标志中最重要的是：

- `IRQF_DISABLED`——该标志被设置后，意味着内核在处理中断处理程序本身期间，要禁止所有的其他中断。如果不设置，中断处理程序可以与除本身外的其他任何中断同时运行。多数中断处理程序是不会去设置该位的，因为禁止所有中断是一种野蛮行为。这种用法留给希望快速执行的轻量级中断。这一标志是 `SA_INTERRUPT` 标志的当前表现形式，在过去的中断中用以区分“快速”和“慢速”中断。
- `IRQF_SAMPLE_RANDOM`——此标志表明这个设备产生的中断对内核熵池（entropy pool）有贡献。内核熵池负责提供从各种随机事件导出的真正的随机数。如果指定了该标志，那么来自该设备的中断间隔时间就会作为熵填充到熵池。如果你的设备以预知的速率产生中断（如系统定时器），或者可能受外部攻击者（如联网设备）的影响，那么就不要设置这个标志。相反，有其他很多硬件产生中断的速率是不可预知的，所以都能成为一种较好的熵源。
- `IRQF_TIMER`——该标志是特别为系统定时器的中断处理而准备的。
- `IRQF_SHARED`——此标志表明可以在多个中断处理程序之间共享中断线。在同一个给定线上注册的每个处理程序必须指定这个标志；否则，在每条线上只能有一个处理程序。有关共享中断处理程序的更多信息将在下面的内容中提供。

第四个参数 `name` 是与中断相关的设备的 ASCII 文本表示。例如，PC 机上键盘中断对应的这个值为“keyboard”。这些名字会被 `/proc/irq` 和 `/proc/interrupts` 文件使用，以便与用户通信，稍后我们将对此进行简短讨论。

第五个参数 `dev` 用于共享中断线。当一个中断处理程序需要释放时（稍后讨论），`dev` 将提供唯一的标志信息（cookie），以便从共享中断线的诸多中断处理程序中删除指定的那一个。如果没有这个参数，那么内核不可能知道在给定的中断线上到底要删除哪一个处理程序。如果无须共享中断线，那么将该参数赋为空值（NULL）就可以了，但是，如果中断线是被共享的，那么就必须传递唯一的信息（除非设备又旧又破且位于 ISA 总线上，那么就必须支持共享中断）。另

外，内核每次调用中断处理程序时，都会把这个指针传递给它^①。实践中往往会通过它传递驱动程序的设备结构：这个指针是唯一的，而且有可能在中断处理程序内被用到。

`request_irq()` 成功执行会返回 0。如果返回非 0 值，就表示有错误发生，在这种情况下，指定的中断处理程序不会被注册。最常见的错误是 `-EBUSY`，它表示给定的中断线已经在使用（或者当前用户或者你没有指定 `IRQF_SHARED`）。

注意，`request_irq()` 函数可能会睡眠，因此，不能在中断上下文或其他不允许阻塞的代码中调用该函数。天真地在睡眠不安全的上下文中调用 `request_irq()` 函数，是一种常见错误。造成这种错误的部分原因是为什么 `request_irq()` 会引起堵塞——这确实让人费解。在注册的过程中，内核需要在 `/proc/irq` 文件中创建一个与中断对应的项。函数 `proc_mkdir()` 就是用来创建这个新的 `procfs` 项的。`proc_mkdir()` 通过调用函数 `proc_create()` 对这个新的 `procs` 项进行设置，而 `proc_create()` 会调用函数 `kmalloc()` 来请求分配内存。我们在第 12 章中将会看到，函数 `kmalloc()` 是可以睡眠的。看清楚了，你的程序就是跑到那里小憩去了！

7.4.2 一个中断例子

在一个驱动程序中请求一个中断线，并在通过 `request_irq()` 安装中断处理程序：

```
request_irq():
if (request_irq(irqn, my_interrupt, IRQF_SHARED, "my_device", my_dev)) {
    printk(KERN_ERR "my_device: cannot register IRQ %d\n", irqn);
    return -EIO;
}
```

在这个例子中，`irqn` 是请求的中断线；`my_interrupt` 是中断处理程序；我们通过标志设置中断线可以共享；设备命名为“`my_device`”；最后是传递 `my_dev` 变量给 `dev` 形参。如果请求失败，那么这段代码将打印出一个错误并返回。如果调用返回 0，则说明处理程序已经成功安装。此后，处理程序就会在响应该中断时被调用。有一点很重要，初始化硬件和注册中断处理程序的顺序必须正确，以防止中断处理程序在设备初始化完成之前就开始执行。

7.4.3 释放中断处理程序

卸载驱动程序时，需要注销相应的中断处理程序，并释放中断线。上述动作需要调用：

```
void free_irq(unsigned int irq, void *dev)
```

如果指定的中断线不是共享的，那么，该函数删除处理程序的同时将禁用这条中断线。如果中断线是共享的，则仅删除 `dev` 所对应的处理程序，而这条中断线本身只有在删除了最后一个处理程序时才会被禁用。由此可以看出为什么唯一的 `dev` 如此重要。对于共享的中断线，需要一个唯一的信息来区分其上面的多个处理程序，并让 `free_irq()` 仅仅删除指定的处理程序。不管在哪

① 中断处理程序都是预先在内核进行注册的回调函数（callback function），而不同的函数位于不同的驱动程序中，所以在这些函数共享同一个中断线时，内核必须准确地为它们创造执行环境，此时就可以通过这个指针将有用的环境信息传递给它们了。——译者注

种情况下（共享或不共享），如果 dev 非空，它都必须与需要删除的处理程序相匹配。必须从进程上下文中调用 `free_irq()`。

表 7-1 给出了终端处理函数的注册和注销函数。

表 7-1 中断注册方法表

函 数	描 述
<code>request_irq()</code>	在给定的中断线上注册一给定的中断处理程序
<code>free_irq()</code>	如果在给定的中断线上没有中断处理程序，则注销响应的处理程序，并禁用其中断线

7.5 编写中断处理程序

以下是一个中断处理程序声明：

```
static irqreturn_t intr_handler(int irq, void *dev)
```

注意，它的类型与 `request_irq()` 参数中 handler 所要求的参数类型相匹配。第一个参数 `irq` 就是这个处理程序要响应的中断的中断号。如今，这个参数已经没有太大用处了，可能只是在打印日志信息时会用到。而在 2.0 版以前的 Linux 内核中，由于没有 dev 这个参数，必须通过 `irq` 才能区分使用相同驱动程序，因而也使用相同的中断处理程序的多个设备。例如，具有多个相同类型硬盘驱动控制器的计算机。

第二个参数 dev 是一个通用指针，它与在中断处理程序注册时传递给 `request_irq()` 的参数 dev 必须一致。如果该值有唯一确定性（这样做是为了能支持共享），那么它就相当于一个 cookie，可以用来区分共享同一中断处理程序的多个设备。另外 dev 也可能指向中断处理程序使用的一个数据结构。因为对每个设备而言，设备结构都是唯一的，而且可能在中断处理程序中也用得到，因此，它也通常被看做 dev。

中断处理程序的返回值是一个特殊类型：`irqreturn_t`。中断处理程序可能返回两个特殊的值：`IRQ_NONE` 和 `IRQ_HANDLED`。当中断处理程序检测到一个中断，但该中断对应的设备并不是在注册处理函数期间指定的产生源时，返回 `IRQ_NONE`；当中断处理程序被正确调用，且确实是它所对应的设备产生了中断时，返回 `IRQ_HANDLED`。另外，也可以使用宏 `IRQ_RETVAL(val)`。如果 val 为非 0 值，那么该宏返回 `IRQ_HANDLED`；否则，返回 `IRQ_NONE`。利用这些特殊的值，内核可以知道设备发出的是否是一种虚假的（未请求）中断。如果给定中断线上所有中断处理程序返回的都是 `IRQ_NONE`，那么，内核就可以检测到出了问题。注意，`irqreturn_t` 这个返回类型实际上就是一个 `int` 型。之所以使用这些特殊值是为了与早期的内核保持兼容——2.6 版之前的内核并不支持这种特性，中断处理程序只需返回 `void` 就行了。如果要在 2.4 或更早的内核上使用这样的驱动程序，只需简单地将 `typedef irqreturn_t` 改为 `void`，屏蔽掉此特性，并给 `no-ops` 定义不同的返回值，其他用不着做什么大的修改。中断处理程序通常会标记为 `static`，因为它从来不会被别的文件中的代码直接调用。

中断处理程序扮演什么样的角色要取决于产生中断的设备和该设备为什么要发送中断。即使其他什么工作也不做，绝大部分的中断处理程序至少需要知道产生中断的设备，告诉它已经收到中断了。对于复杂一些的设备，可能还需要在中断处理程序中发送和接收数据，以及执行一

些扩充的工作。如前所述，应尽可能将扩充的工作推给下半部处理程序，这点将在第 8 章中进行讨论。

重入和中断处理程序

Linux 中的中断处理程序是无须重入的。当一个给定的中断处理程序正在执行时，相应的中断线在所有处理器上都会被屏蔽掉，以防止在同一中断线上接收另一个新的中断。通常情况下，所有其他的中断都是打开的，所以这些不同中断线上的其他中断都能被处理，但当前中断线总是被禁止的。由此可以看出，同一个中断处理程序绝对不会被同时调用以处理嵌套的中断。这极大地简化了中断处理程序的编写。

7.5.1 共享的中断处理程序

共享的处理程序与非共享的处理程序在注册和运行方式上比较相似，但差异主要有以下三处：

- `request_irq()` 的参数 `flags` 必须设置 `IRQF_SHARED` 标志。
- 对于每个注册的中断处理程序来说，`dev` 参数必须唯一。指向任一设备结构的指针就可以满足这一要求；通常会选择设备结构，因为它是唯一的，而且中断处理程序可能会用到它。不能给共享的处理程序传递 `NULL` 值。
- 中断处理程序必须能够区分它的设备是否真的产生了中断。这既需要硬件的支持，也需要处理程序中有相关的处理逻辑。如果硬件不支持这一功能，那中断处理程序肯定会束手无策，它根本没法知道到底是与它对应的设备发出了这个中断，还是共享这条中断线的其他设备发出了这个中断。

所有共享中断线的驱动程序都必须满足以上要求。只要有任何一个设备没有按规则进行共享，那么中断线就无法共享了。指定 `IRQF_SHARED` 标志以调用 `request_irq()` 时，只有在以下两种情况下才可能成功：中断线当前未被注册，或者在该线上的所有已注册处理程序都指定了 `IRQF_SHARED`。注意，在这一点上 2.6 版与以前的内核是不同的，共享的处理程序可以混用 `IRQF_DISABLED`。

内核接收一个中断后，它将依次调用在该中断线上注册的每一个处理程序。因此，一个处理程序必须知道它是否应该为这个中断负责。如果与它相关的设备并没有产生中断，那么处理程序应该立即退出。这需要硬件设备提供状态寄存器（或类似机制），以便中断处理程序进行检查。毫无疑问，大多数硬件都提供这种功能。

7.5.2 中断处理程序实例

让我们考察一个实际的中断处理程序，它来自 `real-time clock (RTC)` 驱动程序，可以在 `drivers/char/rtc.c` 中找到。很多机器（包括 PC）都可以找到 RTC。它是一个从系统定时器中独立出来的设备，用于设置系统时钟，提供报警器（alarm）或周期性的定时器。对大多数体系结构而言，系统时钟的设置，通常只需要向某个特定的寄存器或 I/O 地址写入想要的时间就可以了。然而报警器或周期性定时器通常就得靠中断来实现。这种中断与生活中的闹铃差不多：中断发出

时，报警器或定时器就会启动。

RTC 驱动程序装载时，`rtc_init()` 函数会被调用，对这个驱动程序进行初始化。它的职责之一就是注册中断处理程序：

```
/* 对 rtc_irq 注册 rtc_interrupt */
if (request_irq(rtc_irq, rtc_interrupt, IRQF_SHARED, "rtc", (void *)&rtc_port)) {
    printk(KERN_ERR "rtc: cannot register IRQ %d\n", rtc_irq);
    return -EIO;
}
```

从中我们看到，中断号由 `rtc_irq` 指定。这个变量用于为给定体系结构指定 RTC 中断。例如，在 PC 上，RTC 位于 IRQ 8。第二个参数是我们的中断处理程序 `rtc_interrupt`——它将与其它中断处理程序共享中断线，因为它设置了 `IRQF_SHARED` 标志。由第四个参数我们看出，驱动程序的名称为“rtc”。因为这个设备允许共享中断线，所以它给 `dev` 型参传递了一个面向每个设备的实参值。

最后要展示的是处理程序本身：

```
static irqreturn_t rtc_interrupt(int irq, void *dev)
{
    /*
     * 可以是报警器中断、更新完成的中断或周期性中断
     * 我们把状态保存在 rtc_irq_data 的低字节中，
     * 而把从最后一次读取之后所接收的中断号保存在其余字节中
     */
    spin_lock (&rtc_lock);

    rtc_irq_data += 0x100;
    rtc_irq_data &= ~0xff;
    rtc_irq_data |= (CMOS_READ(RTC_INTR_FLAGS) & 0xF0);

    if (rtc_status & RTC_TIMER_ON)
        mod_timer(&rtc_irq_timer, jiffies + HZ/rtc_freq + 2*HZ/100);

    spin_unlock (&rtc_lock);

    /*
     * 现在执行其余的操作
     */
    spin_lock(&rtc_task_lock);
    if (rtc_callback)
        rtc_callback->func(rtc_callback->private_data);
    spin_unlock(&rtc_task_lock);
    wake_up_interruptible(&rtc_wait);

    kill_fasync (&rtc_async_queue, SIGIO, POLL_IN);

    return IRQ_HANDLED;
}
```

只要计算机一接收到 RTC 中断，就会调用这个函数。首先要注意的是使用了自旋锁——第一次调用是为了保证 `rtc_irq_data` 不被 SMP 机器上的其他处理器同时访问，第二次调用避免 `rtc_`

callback 出现相同的情况。锁机制在第 10 章中进行讨论。

rtc_irq_data 变量是无符号长整数，存放有关 RTC 的信息，每次中断时都会更新以反映中断的状态。

接下来，如果设置了 RTC 周期性定时器，就要通过函数 mod_timer() 对其更新。定时器在第 11 章进行讨论。

代码的最后一部分——处于注释“现在执行其余的操作”下，会执行一个可能被预先设置好的回调函数。RTC 驱动程序允许注册一个回调函数，并在每个 RTC 中断到来时执行。

最后，这个函数会返回 IRQ_HANDLED，表明已经正确地完成了对此设备的操作。因为这个中断处理程序不支持共享，而且 RTC 也没有什么用来测试虚假中断的机制，所以该处理程序总是返回 IRQ_HANDLED。

7.6 中断上下文

当执行一个中断处理程序时，内核处于中断上下文（interrupt context）中。让我们先回忆一下进程上下文。进程上下文是一种内核所处的操作模式，此时内核代表进程执行——例如，执行系统调用或运行内核线程。在进程上下文中，可以通过 current 宏关联当前进程。此外，因为进程是以进程上下文的形式连接到内核中的，因此，进程上下文可以睡眠，也可以调用调度程序。

与之相反，中断上下文和进程并没有什么瓜葛。与 current 宏也是不相干的（尽管它会指向被中断的进程）。因为没有后备进程，所以中断上下文不可以睡眠，否则又怎能再对它重新调度呢？因此，不能从中断上下文中调用某些函数。如果一个函数睡眠，就不能在你的中断处理程序中使用它——这是对什么样的函数可以在中断处理程序中使用的限制。

中断上下文具有较为严格的时间限制，因为它打断了其他代码。中断上下文中的代码应当迅速、简洁，尽量不要使用循环去处理繁重的工作。有一点非常重要，请永远牢记：中断处理程序打断了其他的代码（甚至可能是打断了在其他中断线上的另一中断处理程序）。正是因为这种异步执行的特性，所以所有的中断处理程序必须尽可能的迅速、简洁。尽量把工作从中断处理程序中分离出来，放在下半部来执行，因为下半部可以在更合适的时间运行。

中断处理程序栈的设置是一个配置选项。曾经，中断处理程序并不具有自己的栈。相反，它们共享所中断进程的内核栈^①。内核栈的大小是两页，具体地说，在 32 位体系结构上是 8KB，在 64 位体系结构上是 16KB。因为在这种设置中，中断处理程序共享别人的堆栈，所以它们在栈中获取空间时必须非常节约。当然，内核栈本来就有限，因此，所有的内核代码都应该谨慎利用它。

在 2.6 版早期的内核中，增加了一个选项，把栈的大小从两页减到一页，也就是在 32 位的系统上只提供 4KB 的栈。这就减轻了内存的压力，因为系统中每个进程原先都需要两页连续，且不可换出的内核内存。为了应对栈大小的减少，中断处理程序拥有了自己的栈，每个处理器一个，大小为一页。这个栈就称为中断栈，尽管中断栈的大小是原先共享栈的一半，但平均可用栈

① 总得有一个进程在运行着。当没有进程可调度时，空任务运行。

空间大得多，因为中断处理程序把这一整页占为己有。

你的中断处理程序不必关心栈如何设置，或者内核栈的大小是多少。总而言之，尽量节约内核栈空间。

7.7 中断处理机制的实现

中断处理系统在 Linux 中的实现是非常依赖于体系结构的，想必你对此不会感到特别惊讶。实现依赖于处理器、所使用的中断控制器的类型、体系结构的设计及机器本身。

图 7-1 是中断从硬件到内核的路由。设备产生中断，通过总线把电信号发送给中断控制器。如果中断线是激活的（它们是允许被屏蔽的），那么中断控制器就会把中断发往处理器。在大多数体系结构中，这个工作就是通过电信号给处理器的特定管脚发送一个信号。除非在处理器上禁止该中断，否则，处理器会立即停止它正在做的事，关闭中断系统，然后跳到内存中预定义的位置开始执行那里的代码。这个预定义的位置是由内核设置的，是中断处理程序的入口点。

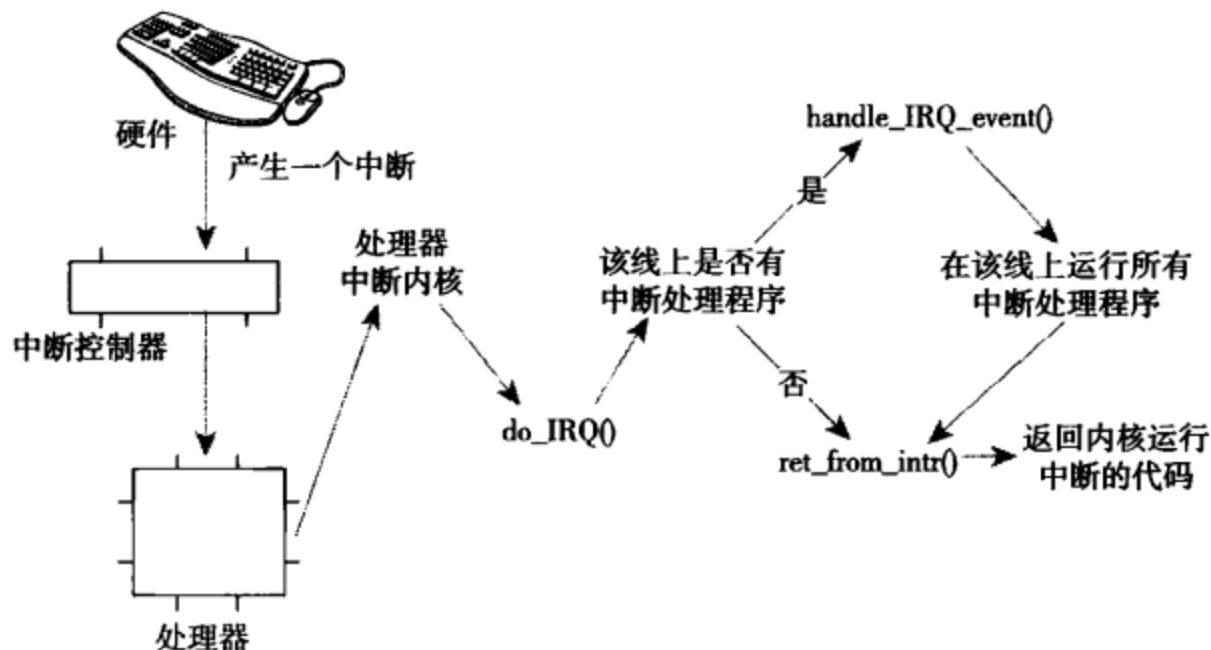


图 7-1 中断从硬件到内核的路由

在内核中，中断的旅程开始于预定义入口点，这类似于系统调用通过预定义的异常句柄进入内核。对于每条中断线，处理器都会跳到对应的一个唯一的位置。这样，内核就可知道所接收中断的 IRQ 号了。初始入口点只是在栈中保存这个号，并存放当前寄存器的值（这些值属于被中断的任务）；然后，内核调用函数 `do_IRQ()`。从这里开始，大多数中断处理代码是用 C 编写的——但它们依然与体系结构相关。

`do_IRQ()` 的声明如下：

```
unsigned int do_IRQ(struct pt_regs regs)
```

因为 C 的调用惯例是要把函数参数放在栈的顶部，因此 `pt_regs` 结构包含原始寄存器的值，这些值是以前在汇编入口例程中保存在栈中的。中断的值也会得以保存，所以，`do_IRQ()` 可以将它提取出来。

计算出中断号后，`do_IRQ()` 对所接收的中断进行应答，禁止这条线上的中断传递。在普通的 PC 机上，这些操作是由 `mask_and_ack_8259A()` 来完成的。

接下来，do_IRQ() 需要确保在这条中断线上有一个有效的处理程序，而且这个程序已经启动，但是当前并没有执行。如果是这样的话，do_IRQ() 就调用 handle_IRQ_event() 来运行这条中断线所安装的中断处理程序。handle_IRQ_event() 方法被定义在文件 kernel/irq/handler.c 中。

```
/**
 * handle_IRQ_event - irq action chain handler
 * @irq:      the interrupt number
 * @action:   the interrupt action chain for this irq
 *
 * Handles the action chain of an irq event
 */
irqreturn_t handle_IRQ_event(unsigned int irq, struct irqaction *action)
{
    irqreturn_t ret, retval = IRQ_NONE;
    unsigned int status = 0;

    if (!(action->flags & IRQF_DISABLED))
        local_irq_enable_in_hardirq();

    do {
        trace_irq_handler_entry(irq, action);
        ret = action->handler(irq, action->dev_id);
        trace_irq_handler_exit(irq, action, ret);

        switch (ret) {
        case IRQ_WAKE_THREAD:
            /*
             * 把返回值设置为已处理，以便可疑的检查不再触发
             */
            ret = IRQ_HANDLED;

            /*
             * 捕获返回值为 WAKE_THREAD 的驱动程序，但是并不创建一个线程函数
             */
            if (unlikely(!action->thread_fn)) {
                warn_no_thread(irq, action);
                break;
            }

            /*
             * 为这次中断唤醒处理线程。万一线程崩溃且被杀死，我们仅仅假装已经处理了该中
             * 断。上述的硬件中断 (hardirq) 处理程序已经禁止设备中断，因此杜绝 irq 产生
             */
            if (likely(!test_bit(IRQTF_DIED,
                                &action->thread_flags))) {
                set_bit(IRQTF_RUNTHREAD, &action->thread_flags);
                wake_up_process(action->thread);
            }

            /* Fall through to add to randomness */
        case IRQ_HANDLED:
```

```

        status |= action->flags;
        break;

    default:
        break;
}

retval |= ret;
action = action->next;
} while (action);

if (status & IRQF_SAMPLE_RANDOM)
    add_interrupt_randomness(irq);
local_irq_disable();

return retval;
}

```

首先，因为处理器禁止中断，这里要把它们打开，就必须在处理程序注册期间指定 `IRQF_DISABLED` 标志。回想一下，`IRQF_DISABLED` 表示处理程序必须在中断禁止的情况下运行。接下来，每个潜在的处理程序在循环中依次执行。如果这条线不是共享的，第一次执行后就退出循环。否则，所有的处理程序都要被执行。之后，如果在注册期间指定了 `IRQF_SAMPLE_RANDOM` 标志，则还要调用函数 `add_interrupt_randomness()`。这个函数使用中断间隔时间为随机数产生器产生熵。最后，再将中断禁止（`do_IRQ()` 期望中断一直是禁止的），函数返回。回到 `do_IRQ()`，该函数做清理工作并返回到初始入口点，然后再从这个入口点跳到函数 `ret_from_intr()`。

`ret_from_intr()` 例程类似于初始入口代码，以汇编语言编写。这个例程检查重新调度是否正在挂起（回想一下第4章，这意味着设置了 `need_resched`）。如果重新调度正在挂起，而且内核正在返回用户空间（也就是说，中断了用户进程），那么，`schedule()` 被调用。如果内核正在返回内核空间（也就是说，中断了内核本身），只有在 `preempt_count` 为 0 时，`schedule()` 才会被调用，否则，抢占内核便是不安全的。在 `schedule()` 返回之后，或者如果没有挂起的工作，那么，原来的寄存器被恢复，内核恢复到曾经中断的点。

在 x86 上，初始的汇编例程位于 `arch/x86/kernel/entry_64.S`（文件 `entry_32.S` 对应 32 位的 x86 体系架构），C 方法位于 `arch/x86/kernel/irq.c`。其他所支持的结构与此类似。

7.8 /proc/interrupts

`procfs` 是一个虚拟文件系统，它只存在于内核内存，一般安装于 `/proc` 目录。在 `procfs` 中读写文件都要调用内核函数，这些函数模拟从真实文件中读或写。与此相关的例子是 `/proc/interrupts` 文件，该文件存放的是系统中与中断相关的统计信息。下面是从单处理器 PC 上输出的信息：

```

CPU0
0:  3602371  XT-PIC  timer
1:  3048     XT-PIC  i8042
2:  0        XT-PIC  cascade
4:  2689466  XT-PIC  uhci-hcd, eth0
5:  0        XT-PIC  EMU10K1
12: 85077    XT-PIC  uhci-hcd
15: 24571    XT-PIC  aic7xxx
NMI: 0
LOC: 3602236
ERR: 0

```

第 1 列是中断线。在这个系统中，现有的中断号为 0 ~ 2、4、5、12 及 15。这里没有显示没有安装处理程序的中断线。第 2 列是一个接收中断数目的计数器。事实上，系统中的每个处理器都存在这样的列，但是，这个机器只有一个处理器。我们看到，时钟中断已接收 3602371 次中断^①，这里，声卡（EMU10K1）没有接收一次中断（这表示机器启动以来还没有使用它）。第 3 列是处理这个中断的中断控制器。XT-PIC 对应于标准的 PC 可编程中断控制器。在具有 I/O APIC 的系统上，大多数中断会列出 IO-APIC-level 或 IO-APIC-edge，作为自己的中断控制器。最后一列是与这个中断相关的设备名字。这个名字是通过参数 devname 提供给函数 request_irq() 的，前面已讨论过了。如果中断是共享的（例子中的 4 号中断就是这种情况），则这条中断线上注册的所有设备都会列出来。

对于想深入探究 procfs 内部的人来说，procfs 代码位于 fs/proc 中。不必惊讶，提供 /proc/interrupts 的函数是与体系结构相关的，叫做 show_interrupts()。

7.9 中断控制

Linux 内核提供了一组接口用于操作机器上的中断状态。这些接口为我们提供了能够禁止当前处理器的中断系统，或屏蔽掉整个机器的一条中断线的能力，这些例程都是与体系结构相关的，可以在 <asm/system.h> 和 <asm/irq.h> 中找到。本章稍后给出的表 7-2 是接口的完整列表。

一般来说，控制中断系统的原因归根结底是需要提供同步。通过禁止中断，可以确保某个中断处理程序不会抢占当前的代码。此外，禁止中断还可以禁止内核抢占。然而，不管是禁止中断还是禁止内核抢占，都没有提供任何保护机制来防止来自其他处理器的并发访问。Linux 支持多处理器，因此，内核代码一般都需要获取某种锁，防止来自其他处理器对共享数据的并发访问。获取这些锁的同时也伴随着禁止本地中断。锁提供保护机制，防止来自其他处理器的并发访问，而禁止中断提供保护机制，则是防止来自其他中断处理程序的并发访问。第 9 章和第 10 章着重讨论同步的各种问题及其对策。因此，必须理解内核中断的控制接口。

7.9.1 禁止和激活中断

用于禁止当前处理器（仅仅是当前处理器）上的本地中断，随后又激活它们的语句为：

^① 作为一个练习，读过第 11 章后，你能在知道时钟产生的中断次数的情况下说出系统已经工作了多久了吗（根据 HZ 值）？知道时钟中断发生了多少次吗？